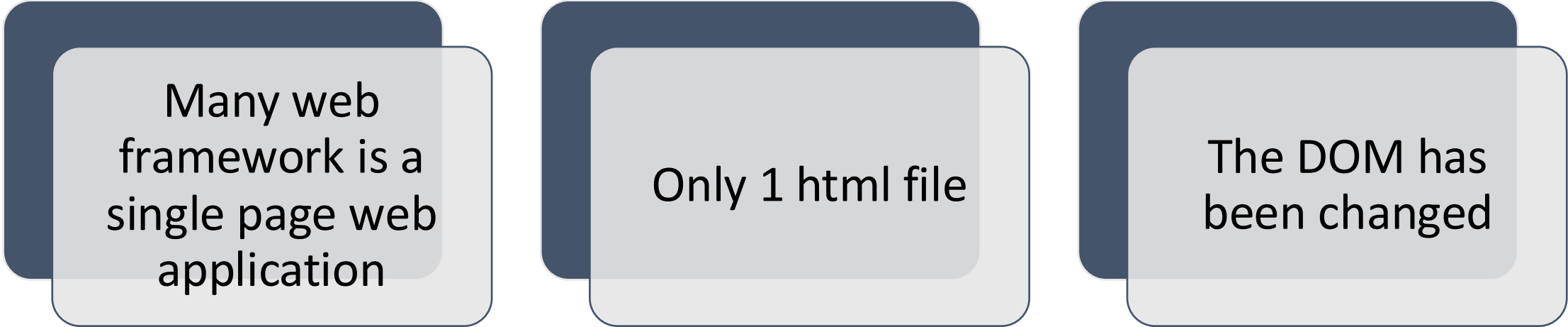


SE 713

Software Backend Development

Introduction to Vue

Single Page Web Application



Many web framework is a single page web application

The diagram consists of three identical, horizontally aligned rectangular boxes. Each box has a dark blue header bar at the top and a light gray body. The text is centered within each box. The boxes are slightly offset from each other, creating a layered effect.

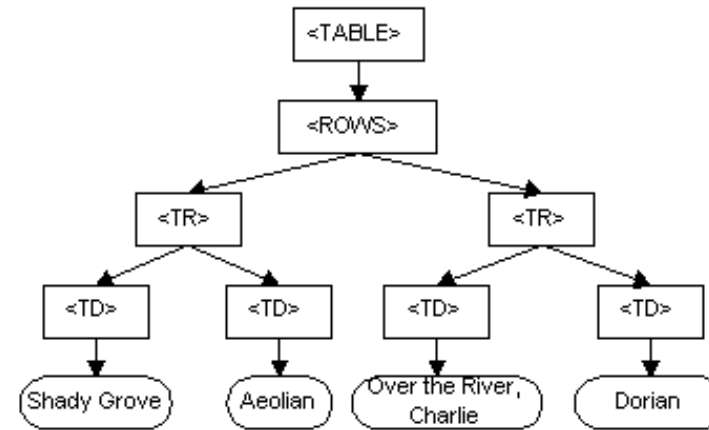
Only 1 html file

The DOM has been changed

DOM

- Document Object Model

```
<TABLE>
  <ROWS>
    <TR>
      <TD>Shady Grove</TD>
      <TD>Aeolian</TD>
    </TR>
    <TR>
      <TD>Over the River, Charlie</TD>
      <TD>Dorian</TD>
    </TR>
  </ROWS>
</TABLE>
```



Single Page Application

Browser manipulate Dom to
visualize

Java script manipulate Dom in
runtime

Vue

Pronounce like View

Progressive framework for building User interface

Core library focused on view layer only

Each Framework provided their own API

- Application Programming Interface
 - How our code will connect to the Framework
 - Send data to the framework back and forth
 - Framework will do the rest

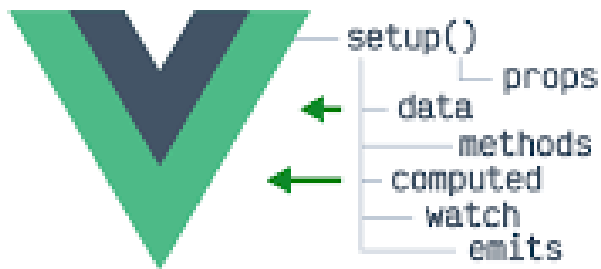


For Vue

OPTIONS API VUE 2 / VUE 3

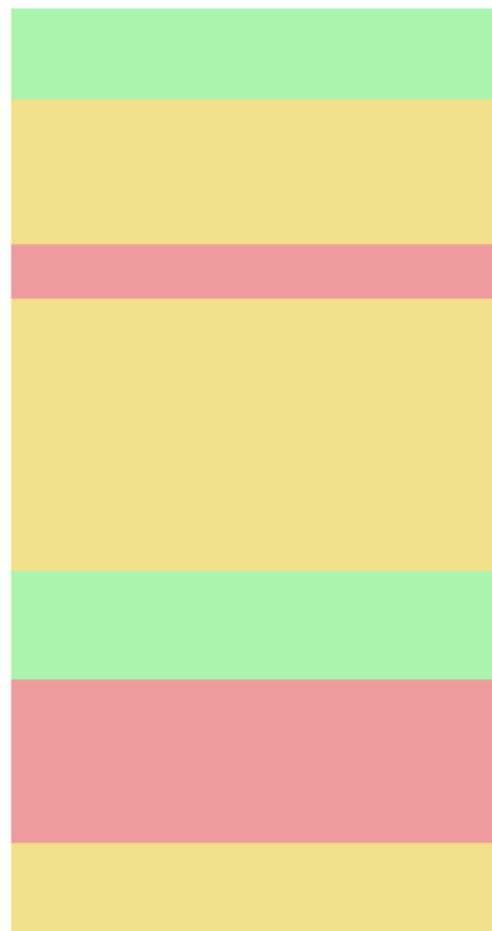


COMPOSITION API VUE 3



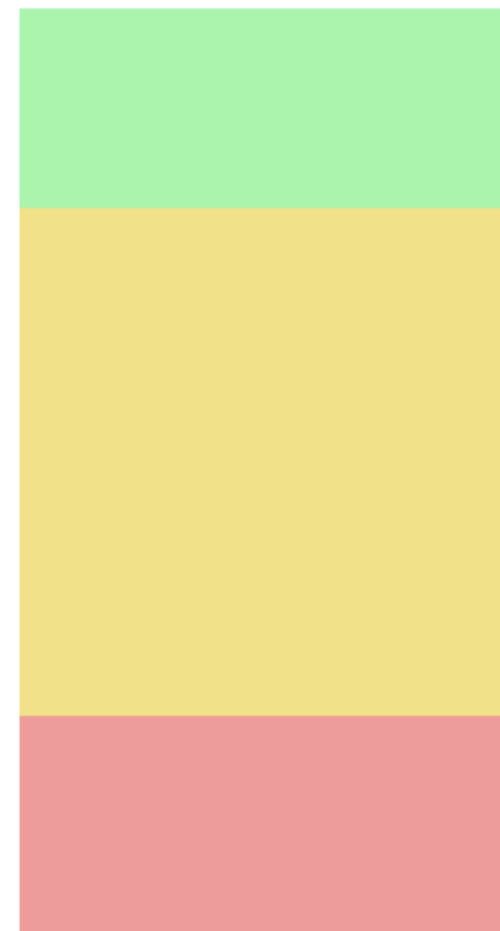
Options API

(code design by **options**)



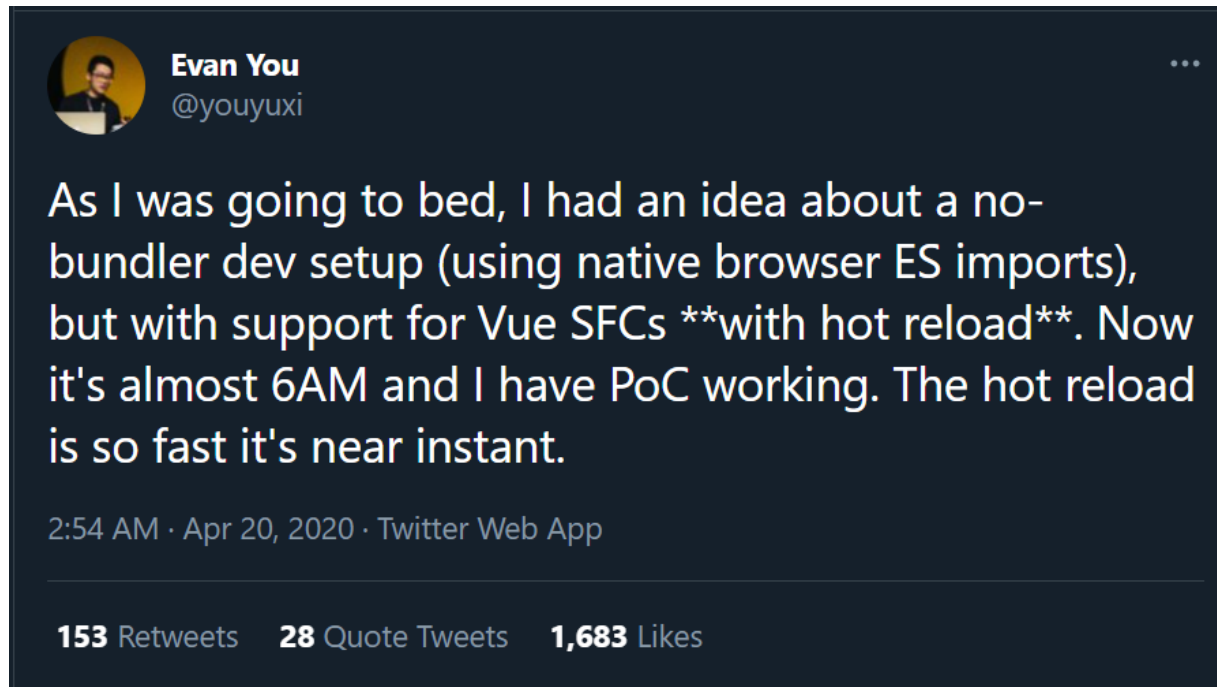
Composition API

(code design by **features**)



What is Vite

- Vite (French word for "quick", pronounced /vit/, like "veet")



Vite

- Next Generation Frontend Tooling

-  Instant Server Start
-  Lightning Fast HMR
-  Rich Features
-  Optimized Build
-  Universal Plugin Interface
-  Fully Typed APIs

Vite

- A dev server that serves your source files over [native ES modules](#), with [rich built-in features](#) and astonishingly fast [Hot Module Replacement \(HMR\)](#).
- A [build command](#) that bundles your code with [Rollup](#), pre-configured to output highly optimized static assets for production.
- Support multiple popular framework
 - Including Vue, react, svelte

Why Vite

- Faster Build times
 - Native ES module
- Optimized Code Size
- Support modern standard

Start Development with Vite

- Try task 1

```
C:\Users\Dto\Desktop\temp>npm create vite@latest
npm WARN config global '--global', '--local' are deprecated. Use '--location=global' instead.
Need to install the following packages:
  create-vite@4.3.2
Ok to proceed? (y) y
? Project name: » real-world-vue|
```

NPM

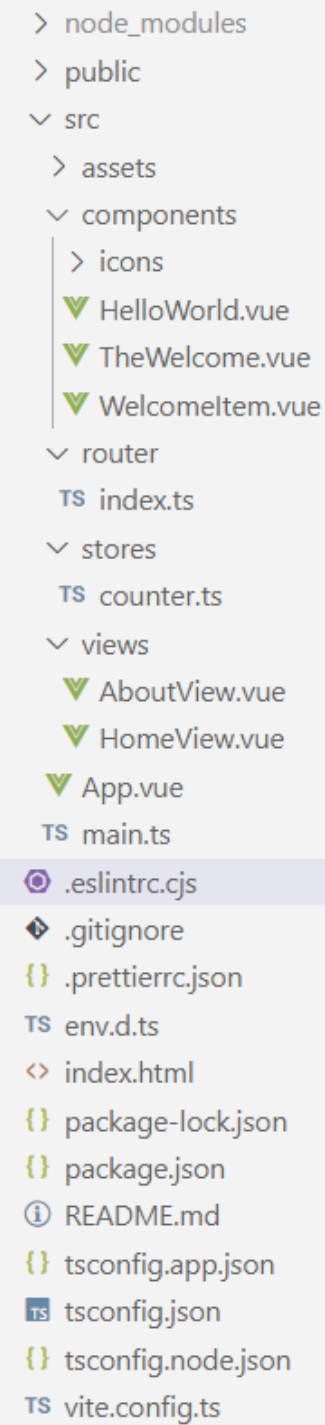
- Package Manager and Installer
- Open-source developers used to share the software
- World Large Software Registry
- Can use any components provided in the registry
 - `npm install package_name`
- Yarn is another package management tool

Lint

- Static code analytic tools
- Use to finding some errors, bug, stylistic error
- Suspect code error
- Available for most language
- Check the code, and the format
- Help the code to be the same convention when multiple developers working together

Take a tour on Vue Project

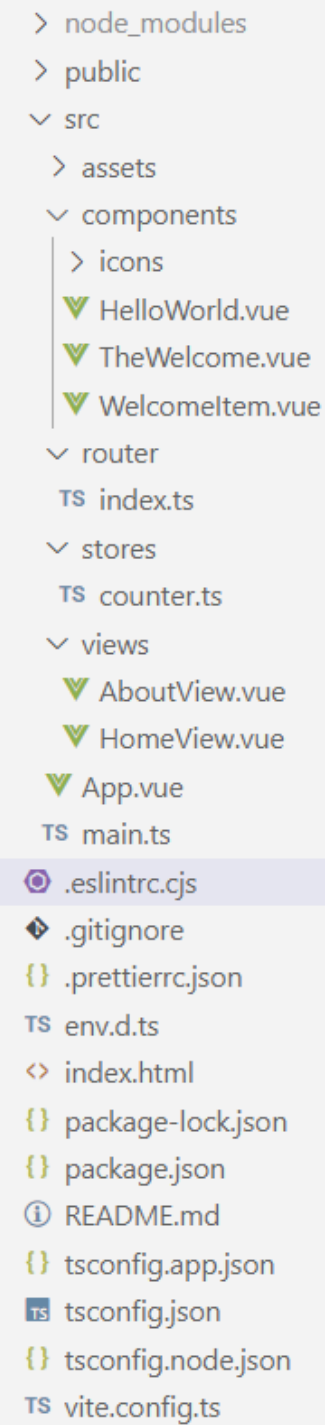
- **node_modules**
 - The library which needed
 - Downloaded when create the Vue project
- **public folder**
 - The static assets, do not run through the build
 - html
- **src folder**
 - The code inside



```
> node_modules
> public
v src
  > assets
  v components
    > icons
    v HelloWorld.vue
    v TheWelcome.vue
    v WelcomeItem.vue
  v router
    TS index.ts
  v stores
    TS counter.ts
  v views
    v AboutView.vue
    v HomeView.vue
    v App.vue
  TS main.ts
.eslintrc.cjs
.gitignore
.prettierrc.json
TS env.d.ts
<> index.html
{} package-lock.json
{} package.json
(i) README.md
{} tsconfig.app.json
TS tsconfig.json
{} tsconfig.node.json
TS vite.config.ts
```

Take a tour on Vue Project

- asset folder
 - Images, fonts which can be optimized
- components folder
 - Store the component
 - Building blocks
- router folder
 - Enable the site navigation
 - Pull the different “views” to show in SPA

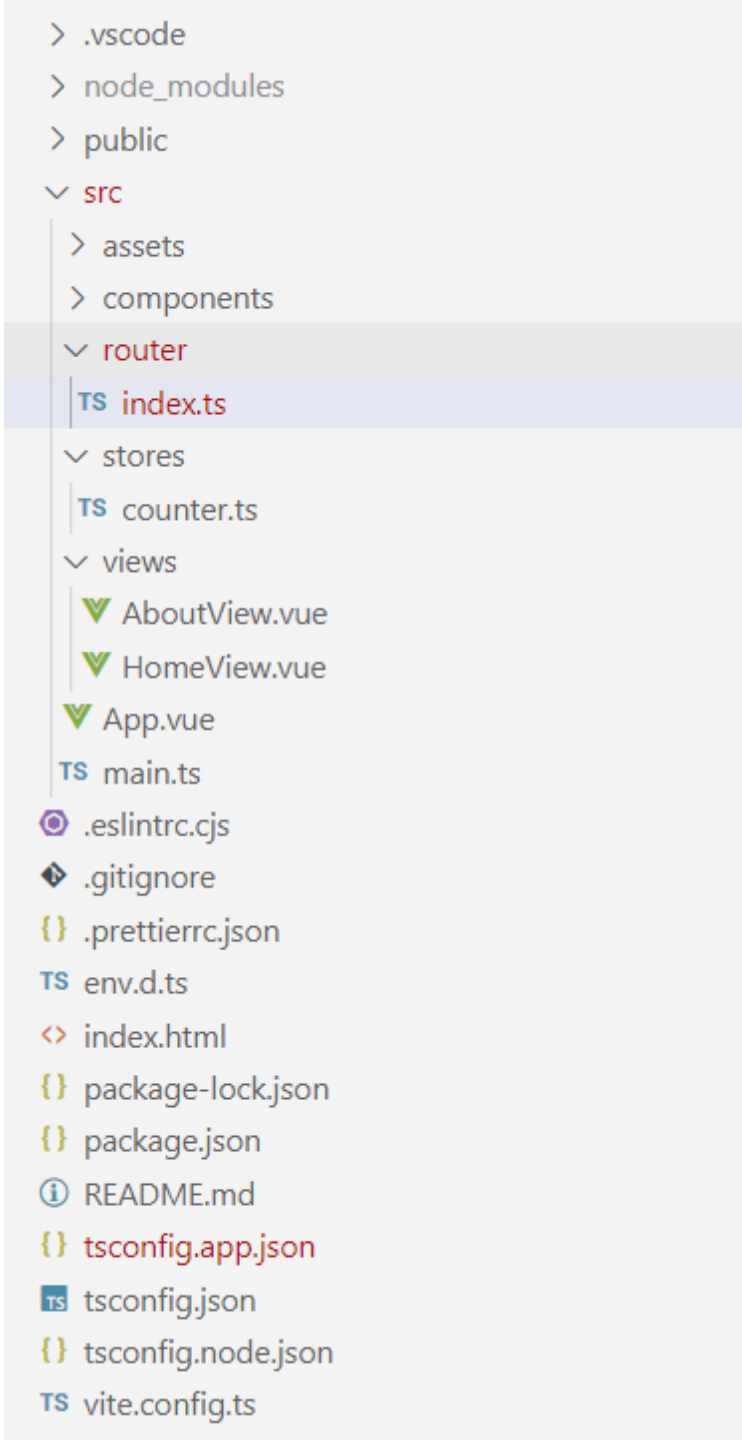


The image shows a file explorer view of a Vue.js project. The structure is as follows:

- > node_modules
- > public
- ✓ src
 - > assets
 - ✓ components
 - > icons
 - ▼ HelloWorld.vue
 - ▼ TheWelcome.vue
 - ▼ WelcomeItem.vue
 - ✓ router
 - TS index.ts
 - ✓ stores
 - TS counter.ts
 - ✓ views
 - ▼ AboutView.vue
 - ▼ HomeView.vue
 - ▼ App.vue
 - TS main.ts
- .eslintrc.cjs (highlighted)
- .gitignore
- .prettierrc.json
- TS env.d.ts
- <> index.html
- package-lock.json
- package.json
- README.md
- tsconfig.app.json
- TS tsconfig.json
- tsconfig.node.json
- TS vite.config.ts

Take a tour on Vue Project

- store folder
 - The Pinia code is here. The state management for Vue
- views folder
 - Component files for different views
- App.vue
 - Root component of all other components
- main.ts
 - The ts file renders the App.vue and mounts to the dom
- .gitignore
 - Define the file which will not be in the git



```
> .vscode
> node_modules
> public
▼ src
  > assets
  > components
  ▼ router
    TS index.ts
  ▼ stores
    TS counter.ts
  ▼ views
    ▼ AboutView.vue
    ▼ HomeView.vue
    ▼ App.vue
    TS main.ts
.eslintrc.cjs
.gitignore
.prettierrc.json
TS env.d.ts
<> index.html
{} package-lock.json
{} package.json
(i) README.md
{} tsconfig.app.json
TS tsconfig.json
{} tsconfig.node.json
TS vite.config.ts
```

The Code Scaffold is created

- The app can be run
 - The customization is required

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <link rel="icon" href="/favicon.ico">
    <meta name="viewport" content="width=device-width,
    <title>Vite App</title>
  </head>
  <body>
    <div id="app"></div>
    <script type="module" src="/src/main.ts"></script>
  </body>
</html>
```

Index.html

App.vue

```
<script setup lang="ts">
import { RouterLink, RouterView } from 'vue-router' 4.9k (gzipped: 2.1k)
import HelloWorld from './components/HelloWorld.vue'
</script>

<template>
  <header>
    
```

```
import './assets/main.css'

import { createApp } from 'vue' 556.4k (gzipped: 178.4k)
import { createPinia } from 'pinia' 4.4k (gzipped: 2.1k)

import App from './App.vue'
import router from './router'

const app = createApp(App)

app.use(createPinia())
app.use(router)

app.mount('#app')
```

main.ts

main.ts

Script setup !!

- a compile-time syntactic sugar for using Composition API inside Single-File Components (SFCs)
- Advantage
 - More succinct code with less boilerplate
 - Ability to declare props and emitted events using pure TypeScript
 - Better runtime performance (the template is compiled into a render function in the same scope, without an intermediate proxy)
 - Better IDE type-inference performance (less work for the language server to extract types from code)

With the script setup

- The vue component can be imported
 - Use as the HTML tag in the code

```
<script setup lang="ts">
import { RouterLink, RouterView } from 'vue-router' 4.9k (gzipped: 2.1k)
import HelloWorld from './components/HelloWorld.vue'
</script>

<template>
  <header>
    

    <div class="wrapper">
      <HelloWorld msg="You did it!" />

      <nav>
        <RouterLink to="/">Home</RouterLink>
        <RouterLink to="/about">About</RouterLink>
      </nav>
    </div>
  </header>

  <RouterView />
</template>
```

App.vue

```
<script setup lang="ts">
defineProps<{
  msg: string
}>()
</script>

<template>
  <div class="greetings">
    <h1 class="green">{{ msg }}</h1>
    <h3>
      You've successfully created a project with
      <a href="https://vitejs.dev/" target="_blank" rel="noopener">Vite</a> +
      <a href="https://vuejs.org/" target="_blank" rel="noopener">Vue 3</a>. What's next?
    </h3>
  </div>
</template>
```

@/components/HelloWorld.vue

What is @?

- In tsconfig.app.json

```
{
  "extends": "@vue/tsconfig/tsconfig.dom.json",
  "include": ["env.d.ts", "src/**/*", "src/**/*.vue"],
  "exclude": ["src/**/__tests__/*"],
  "compilerOptions": {
    "composite": true,
    "tsBuildInfoFile": "./node_modules/.tmp/tsconfig.app.tsbuildinfo",

    "baseUrl": ".",
    "paths": {
      "@/*": ["./src/*"]
    }
  }
}
```

The server can be configured

- For each programmer preference
- Add the correct file and value
 - Override the default value
 - For customize in each computer
- For example
 - vite.config.ts

```
import { fileURLToPath, URL } from 'node:url'

import { defineConfig } from 'vite'
import vue from '@vitejs/plugin-vue' 32.1k (gzipped: 12.6k)
import VueDevTools from 'vite-plugin-vue-devtools'

// https://vitejs.dev/config/
export default defineConfig({
  plugins: [
    vue(),
    VueDevTools(),
  ],
  resolve: {
    alias: {
      '@': fileURLToPath(new URL('./src', import.meta.url))
    }
  }
})
```

Lint setting

- Allow or disallow checking some data
- Create the file `.prettierrc.json`

```
{  
  "$schema": "https://json.schemastore.org/prettierrc",  
  "semi": false,  
  "tabWidth": 2,  
  "singleQuote": true,  
  "printWidth": 100,  
  "trailingComma": "none"  
}
```

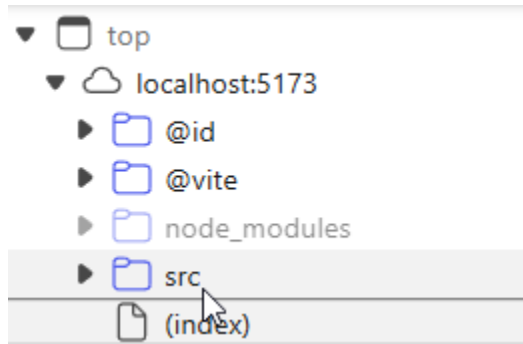
- Set the format of source code
 - Some easy fix
 - Can be fix using `npm run format`

With the npm command

- Check in package.json
- When we run the application
 - npm run dev is called
- The build process is run
 - The code injection is happened
 - The code in the proper folder name is added

```
{ You, last month • first commit
  "name": "real-world-vue",
  "version": "0.0.0",
  "private": true,
  "type": "module",
  ▶ Debug
  "scripts": {
    "dev": "vite",
    "build": "run-p type-check \"build-only {@}\" --",
    "preview": "vite preview",
    "build-only": "vite build",
    "type-check": "vue-tsc --build --force",
    "lint": "eslint . --ext .vue,.js,.jsx,.cjs,.mjs,.ts,.tsx,.cts,.mts --fix --ignore-path .gitignore",
    "format": "prettier --write src/"
  },
  "dependencies": {
    "pinia": "^2.1.7",
    "vue": "^3.4.21",
    "vue-router": "^4.3.0"
  },
}
```


We can check on the output



```
</head>
<body>
  <div id="app"></div>
  <script type="module" src="/src/main.ts"></script>
</body>
</html>
```

- The code is packed in one file

When build

- The code is packed as a js file

vite v5.2.12 building for production...

✓ 48 modules transformed.

dist/index.html	0.44 kB	gzip: 0.29 kB
dist/assets/AboutView-C6Dx7pxG.css	0.09 kB	gzip: 0.10 kB
dist/assets/index-D218GWXz.css	4.21 kB	gzip: 1.30 kB
dist/assets/AboutView-C7nn-kfc.js	0.22 kB	gzip: 0.20 kB
dist/assets/index-D0wvez2B.js	89.23 kB	gzip: 35.29 kB

✓ built in 709ms

- Note that About View is the lazy loading
 - Check in router/index.ts

```
const __vite__fileDeps=["assets/AboutView-C7nn-kfc.js","assets/AboutView-C6Dx7pxG.css"],__vite__mapDeps=i=>i.map(i=>__vite__fileDeps[i]);
(function(){const t=document.createElement("link").relList;if(t&&t.supports&&t.supports("modulepreload"))return;for(const r of document.querySelectorAll('link[rel="modulepreload"]'))s(r);new MutationObserver(r=>{for(const o of r){if(o.type==="childList"for(const i of o.addedNodes)i.tagName==="LINK"&&i.rel==="modulepreload"&&s(i)}}).observe(document,{childList:!0,subtree:!0});function n(r){const o={};return r.integrity&&(o.integrity=r.integrity),r.referrerPolicy&&(o.referrerPolicy=r.referrerPolicy),r.crossOrigin==="use-credentials"?o.credentials="include":r.crossOrigin==="anonymous"?o.credentials="omit":o.credentials="same-origin",o}function s(r){if(r.ep)return;r.ep=!0;const o=n(r);fetch(r.href,o)}()/**
 * @vue/shared v3.4.27
 * (c) 2018-present Yuxi (Evan) You and Vue contributors
 * @license MIT
 **/! #__NO_SIDE_EFFECTS__ */function es(e,t){const n=new Set(e.split(","));return s=>n.has(s)}const ee={},vt=[],Ee=(()=>{}),Vo=(()=>!1,pn=e=>e.charCodeAt(0)===111&&e.charCodeAt(1)===110&&(e.charCodeAt(2)>122||e.charCodeAt(2)<97),ts=e=>e.startsWith("onUpdate:"),le=Object.assign,ns=(e,t)>{const n=e.indexOf(t);n>-1&&e.splice(n,1),ko=Object.prototype.hasOwnProperty,D=(e,t)>=>ko.call(e,t),H=Array.isArray,yt=e=>mn(e)===[object Map],br=e=>mn(e)===[object Set],F=e=>typeof e==="function",ce=e=>typeof e==="string",at=e=>typeof e==="symbol",ne=e=>e!=null&&typeof e==="object",wr=e=>(ne(e)||F(e))&&F(e.then)&&F(e.catch),Er=Object.prototype.toString,mn=e=>Er.call(e),Bo=e=>mn(e).slice(8,-1),xr=e=>mn(e)===[object Object],ss=e=>ce(e)&&e!=="NaN"&&e[0]!=="- "&&""+parseInt(e,10)===e,$t=es("key,ref,ref_for,ref_key,onVnodeBeforeMount,onVnodeMounted,onVnodeBeforeUpdate,onVnodeUpdated,onVnodeBeforeUnmount,onVnodeUnmounted"),gn=e=>{const t=Object.create(null);return n=>t[n]||(t[n]=e(n)),Uo=-/(\w)/g,wt=gn(e=>e.replace(Uo,(t,n)=>n?n.toUpperCase():""),Do=/\B([A-Z])/g,St=gn(e=>e.replace(Do,"-$1").toLowerCase()),Rr=gn(e=>e.charCodeAt(0).toUpperCase()+e.slice(1)),Sn=gn(e=>e?'on${Rr(e)}':""),et=(e,t)=>!Object.is(e,t),Pn=(e,t)=>{for(let n=0;n<e.length;n++)e[n](t)},Cr=(e,t,n,s=!1)=>{Object.defineProperty(e,t,{configurable:!0,enumerable:!1,writable:s,value:n})},Ko=e=>{const t=parseFloat(e);return isNaN(t)?e:t};let Cs;const Sr=(()=>Cs)||Cs=typeof globalThis<"u"?globalThis:typeof self<"u"?self:typeof window<"u"?window:typeof global<"u"?global:{});function rs(e){if(H(e)){const t={};for(let n=0;n<e.length;n++){const s=e[n],r=ce(s)?Yo(s):rs(s);if(r)for(const o in rs(r))if(o!=="__proto__")t[o]=rs(r)[o]}return t}return e}const Uo=/\B([A-Z])/g,Do=/\B([A-Z])/g,St=gn(e=>e.replace(Do,"-$1").toLowerCase()),Rr=gn(e=>e.charCodeAt(0).toUpperCase()+e.slice(1)),Sn=gn(e=>e?'on${Rr(e)}':""),et=(e,t)=>!Object.is(e,t),Pn=(e,t)=>{for(let n=0;n<e.length;n++)e[n](t)},Cr=(e,t,n,s=!1)=>{Object.defineProperty(e,t,{configurable:!0,enumerable:!1,writable:s,value:n})},Ko=e=>{const t=parseFloat(e);return isNaN(t)?e:t};let Cs;const Sr=(()=>Cs)||Cs=typeof globalThis<"u"?globalThis:typeof self<"u"?self:typeof window<"u"?window:typeof global<"u"?global:{});function rs(e){if(H(e)){const t={};for(let n=0;n<e.length;n++){const s=e[n],r=ce(s)?Yo(s):rs(s);if(r)for(const o in rs(r))if(o!=="__proto__")t[o]=rs(r)[o]}return t}return e}const Uo=/\B([A-Z])/g,Do=/\B([A-Z])/g,St=gn(e=>e.replace(Do,"-$1").toLowerCase()),Rr=gn(e=>e.charCodeAt(0).toUpperCase()+e.slice(1)),Sn=gn(e=>e?'on${Rr(e)}':""),et=(e,t)=>!Object.is(e,t),Pn=(e,t)=>{for(let n=0;n<e.length;n++)e[n](t)},Cr=(e,t,n,s=!1)=>{Object.defineProperty(e,t,{configurable:!0,enumerable:!1,writable:s,value:n})},Ko=e=>{const t=parseFloat(e);return isNaN(t)?e:t};let Cs;const Sr=(()=>Cs)||Cs=typeof globalThis<"u"?globalThis:typeof self<"u"?self:typeof window<"u"?window:typeof global<"u"?global:{});function rs(e){if(H(e)){const t={};for(let n=0;n<e.length;n++){const s=e[n],r=ce(s)?Yo(s):rs(s);if(r)for(const o in rs(r))if(o!=="__proto__")t[o]=rs(r)[o]}return t}return e}
```

```
const router = createRouter({
  history: createWebHistory(import.meta.env.BASE_URL),
  routes: [
    {
      path: '/',
      name: 'home',
      component: HomeView
    },
    {
      path: '/about',
      name: 'about',
      // route level code-splitting
      // this generates a separate chunk (About.[hash].js) for this route
      // which is lazy-loaded when the route is visited.
      component: () => import('../views/AboutView.vue')
    }
  ]
})
```

Creating the Component

- EventCard.vue

```
<script setup lang="ts">

</script>

<template>
  <div>
  </div>
</template>

<style scoped>

</style>
```

- No component creation
 - The vue cli will add that code for you
- Try Task 2

The style scope

- The css can be used only in this component
- Reduce the naming conflict with other components
 - The attribute key name is used to define the different

```
▼ <main>
  ▼ <div data-v-f2df5c1a class="event-class">
    ▼ <div data-v-f2df5c1a class="event-card"> == $0
      <span data-v-f2df5c1a>@12:00 on January 28, 2022</span>
      <h4 data-v-f2df5c1a>Cat Adoption Day</h4>
    </div>
  </div>
</main>
```

```
<style scoped>
1 reference
.event-card {
  padding: 20px;
  width: 250px;
  cursor: pointer;
  border: 1px solid #39495c;
  margin-bottom: 18px;
}

1 reference
.event-card:hover {
  transform: scale(1.01);
  box-shadow: 0 3px 12px 0 rgba(0, 0, 0, 0.2);
}
</style>
```

```
.event-card[data-v-f2df5c1a] {
  padding: 20px;
  width: 250px;
  cursor: pointer;
  border: 1px solid #39495c;
  margin-bottom: 18px;
}
```

Separation of Concerns

- Separate the presentation
 - Template
- From the computation
 - Script
- And the declaration
 - Style

Anatomy of a Single File Component

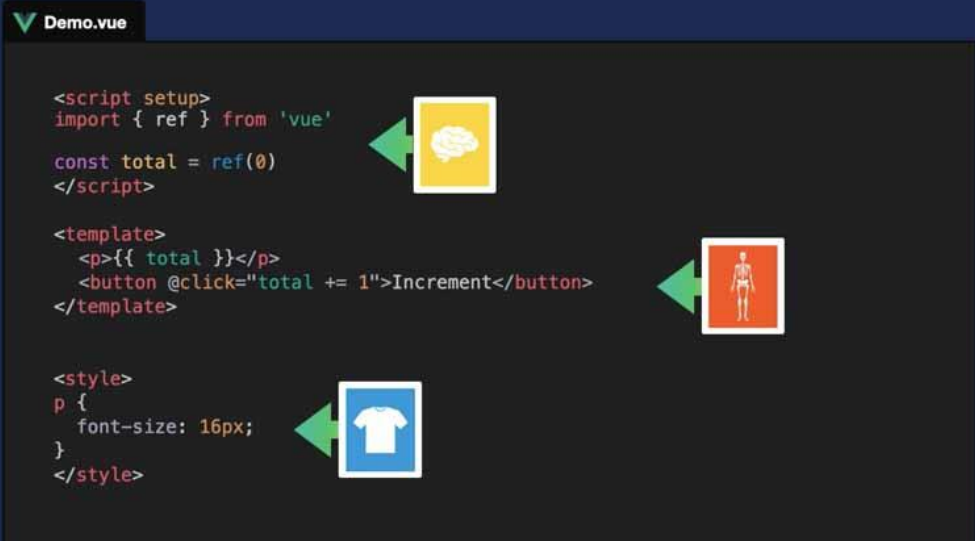
```
✓ Demo.vue

<script setup>
import { ref } from 'vue'

const total = ref(0)
</script>

<template>
  <p>{{ total }}</p>
  <button @click="total += 1">Increment</button>
</template>

<style>
p {
  font-size: 16px;
}
</style>
```



Component can be reused

```
<template>
  <div>
    <EventCard />
    <EventCard />

    <EventCard />
  </div>
</template>
```

Using the condition show

- To show or hide some value
- The if else is provided
 - Regarding to the different framework
- For View
 - v-if v-else is used
 - Try task 3

```
<p v-if="inStock">In Stock</p>  
<p v-else>Out of Stock</p>
```

What happen in Dom

- The element which has not been select has been removed from Dom

```
▼ <div class="product-info">  
  <h1>Shoes</h1>  
  <p>In Stock</p> == $0  
</div>
```


v-show

- The different technique to show/hide the value
- Kept the elements in DOM
 - Hide the elements by add the `display:none;` to the element

```
data() {  
  return {  
    product: 'Shoes',  
    image: './assets/images/socks_green.jpg',  
    inStock: false  
  }  
}
```

```
<div class="product-info">  
  <h1>{{ product }}</h1>  
  <p v-show="inStock">In Stock</p>  
  <p v-else>Out of Stock</p>  
</div>
```

```
<h1>Shoes</h1>  
<p style="display: none;">In Stock</p> == $0  
<p>Out of Stock</p>
```

v-if v-else-if v-else

- The JavaScript Boolean expression can be added to the v-if condition

```
<p v-if="inventory > 10">In Stock</p>
<p v-else>Out of Stock</p>
```

- The chain comparison is also provided

```
<div class="product-info">
  <h1>{{ product }}</h1>
  <p v-if="inventory > 10">In Stock</p>
  <p v-else-if="inventory <= 10 && inventory > 0">In Stock</p>
  <p v-else>Out of Stock</p>
</div>
```

```
setup(){
  const product = ref('Boots')
  const image = ref('./assets/images/socks_green.jpg')
  const inStock = ref(true)
  const inventory = ref(100)
  return {
    product,
    image,
    inStock,
    inventory
  }
}
```

Using the for in command

- Short for -> for each element in array

```
<ul>  
  <li v-for="detail in details">{{ detail }}</li>  
</ul>
```

```
data() {  
  return {  
    product: 'Shoes',  
    image: './assets/images/socks_green.jpg',  
    inStock: true,  
    inventory: 100,  
    details: ['50% cotton', '30% wool', '20% polyester']  
  }  
}
```

Array of Objects

- Objects can be put in an array
- Iterate to get each object
- Access each object value by the key of the objects

```
const variants = ref([  
  { id: 2234, color: 'green' },  
  { id: 2235, color: 'blue' }  
])
```

```
<div v-for="variant in variants" :key="variant.id">  
  {{ variant.color }}  
</div>
```

:key?

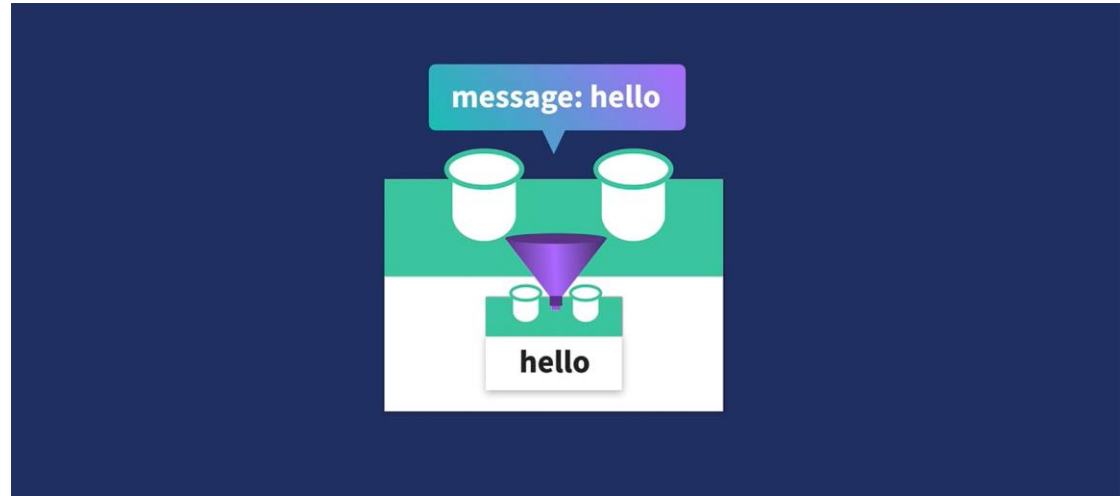
- Create a new key attribute
 - Not standard attribute
 - Use as reference to this elements later in the code

```
▶ <ul>...</ul>  
<div>green</div> == $0  
<div>blue</div>  
</div>
```

- As it is not standard, the key attribute value is not show in the DOM
 - But keep as state of the DOM node in Vue

props element

- Sending the input data
 - From Parent to child
- Help for reuse data



Add props in the child component

```
<script setup lang="ts">
defineProps<{ name: string }>()
</script>
```

- Type is defined
 - Type consist of name and type in the defineProps type
- No import required
- Can use with the data binding

```
<div class="event-class">
  <h1>This is an {{ name }} page</h1>
</div>
```

To send the data

- Props will work as the attribute to use
- Use V-bind to the attribute with the script values

```
<body>
  <div id="app">
    <div class="nav-bar"></div>
    <div class="cart">Cart({{ cart }})</div>
    <product-display :premium="premium"></product-display>
```

- The receive value is a variable
 - Can be used only inside the component
- Try Task 4

Define the Default value

- withDefaults Functions
- When the name is undefined the default values will be used

```
withDefaults(  
  defineProps<{  
    | name: string  
  }>(),  
  {  
    | name: 'default name',  
  },  
)
```

```
mirror_mod = modifier_ob.  
set mirror object to mirror.  
mirror_mod.mirror_object =  
operation == "MIRROR_X":  
mirror_mod.use_x = True  
mirror_mod.use_y = False  
mirror_mod.use_z = False  
operation == "MIRROR_Y":  
mirror_mod.use_x = False  
mirror_mod.use_y = True  
mirror_mod.use_z = False  
operation == "MIRROR_Z":  
mirror_mod.use_x = False  
mirror_mod.use_y = False  
mirror_mod.use_z = True
```

```
selection at the end -add  
mirror_ob.select= 1  
modifier_ob.select=1  
context.scene.objects.active  
("Selected" + str(modifier  
mirror_ob.select = 0  
= bpy.context.selected_object  
data.objects[one.name].select  
print("please select exactly
```

--- OPERATOR CLASSES ---

```
types.Operator):  
X mirror to the selected  
object.mirror_mirror_x"  
mirror X"
```

```
context):  
context.active_object is not
```

One Way Binding

- Bind the data from one side to the other side
 - Data from the JavaScript part to HTML
 - Action from HTML part (user) to Java Script

Data Binding

- Binding the contents in HTML

```
setup(){  
  const product = ref('Socks')  
  return {  
    product  
  }  
}
```

```
<div id="app">  
  <h1>{{product}}</h1>  
</div>
```

What is ref?

- The function to create a reactive object of the input value
- Wrapper Class
- For Reactivity Fundamental



Reactivity

- Vue **tracks** all the ref object
- When the value in the **ref** object is changed
- The DOM will change according to the value which has been changed

As the ref use with const variable

- The address of the object is fixed
- Only the value is changed
- What we need to do is update the value inside
 - In the Javascript part
- In the dom, we can refer to only the name of variable
 - The DOM will get the value of the value automatically

```
// pseudo code, not actual implementation
const myRef = {
  _value: 0,
  get value() {
    track()
    return this._value
  },
  set value(newValue) {
    this._value = newValue
    trigger()
  }
}
```

Not only the value to bind

- Sometimes we need to bind the attribute value

```

```

- Hard to read what has been bound
- Hard for the DOM to read inside as it is the simple string

Introducing the attribute mapping

- v-bind:attribute name
 - v-bind:src
- Link the value in the script part to the attribute value

```
...  
setup(){  
  const product = ref('Boots')  
  const image = ref('./assets/images/socks_green.jpg')  
  return {  
    product,  
    image  
  }  
}
```

```
<div class="product-image">  
    
</div>
```


v-bind:attribute

- May be too long
 - The short hand is :attribute

```
<div class="product-image">  
    
</div>
```

The Global CSS

- Can be placed in App.vue
- Why it is global?

```
<style>
#layout {
  font-family: Avenir, Helvetica, Arial, sans-serif;
  -webkit-font-smoothing: antialiased;
  -moz-osx-font-smoothing: grayscale;
  text-align: center;
  color: ■ #2c3e50;
}
nav {
  padding: 30px;
}
nav a {
  font-weight: bold;
  color: ■ #2c3e50;
}
nav a.router-link-exact-active {
  color: ■ #42b983;
}

h2 {
  font-size: 20px;
}
</style>
```

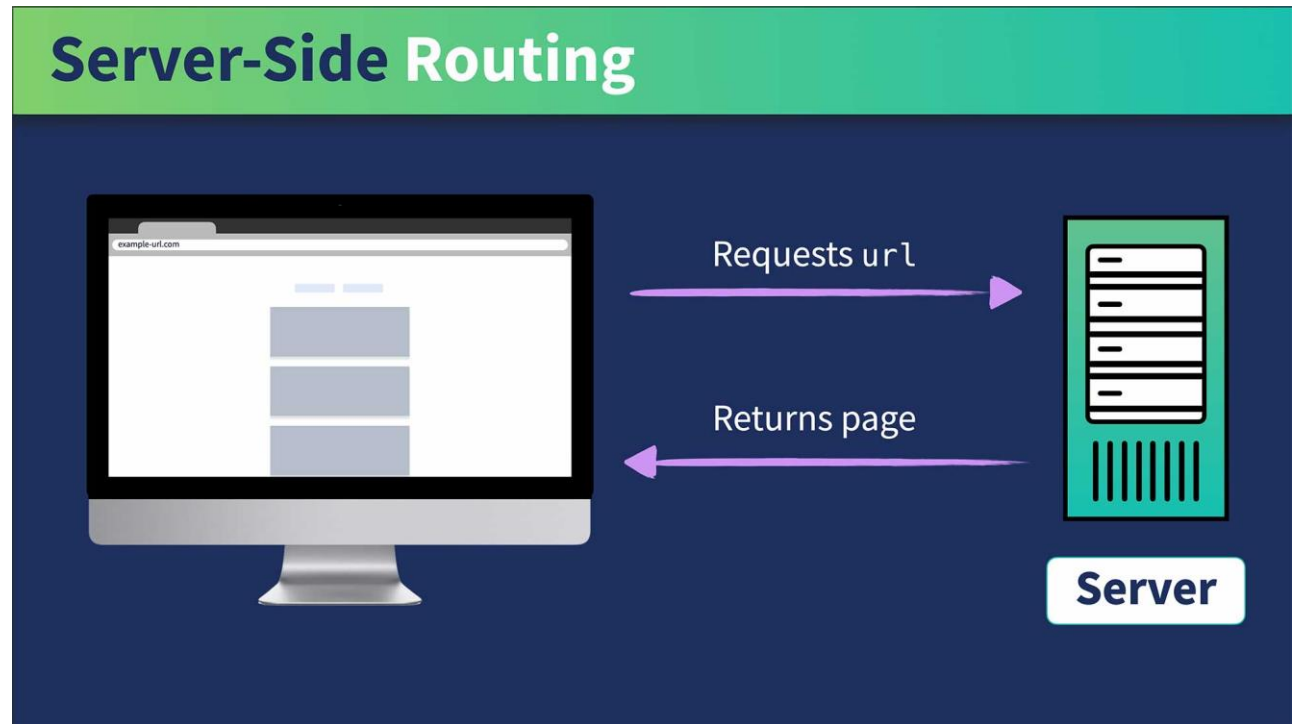
Try Task 5

The Parent and Child Page

- Provide the base component
- The data in route is inserted in the `<RouterView />`
- Show the different location of file, different location of display

The router-view

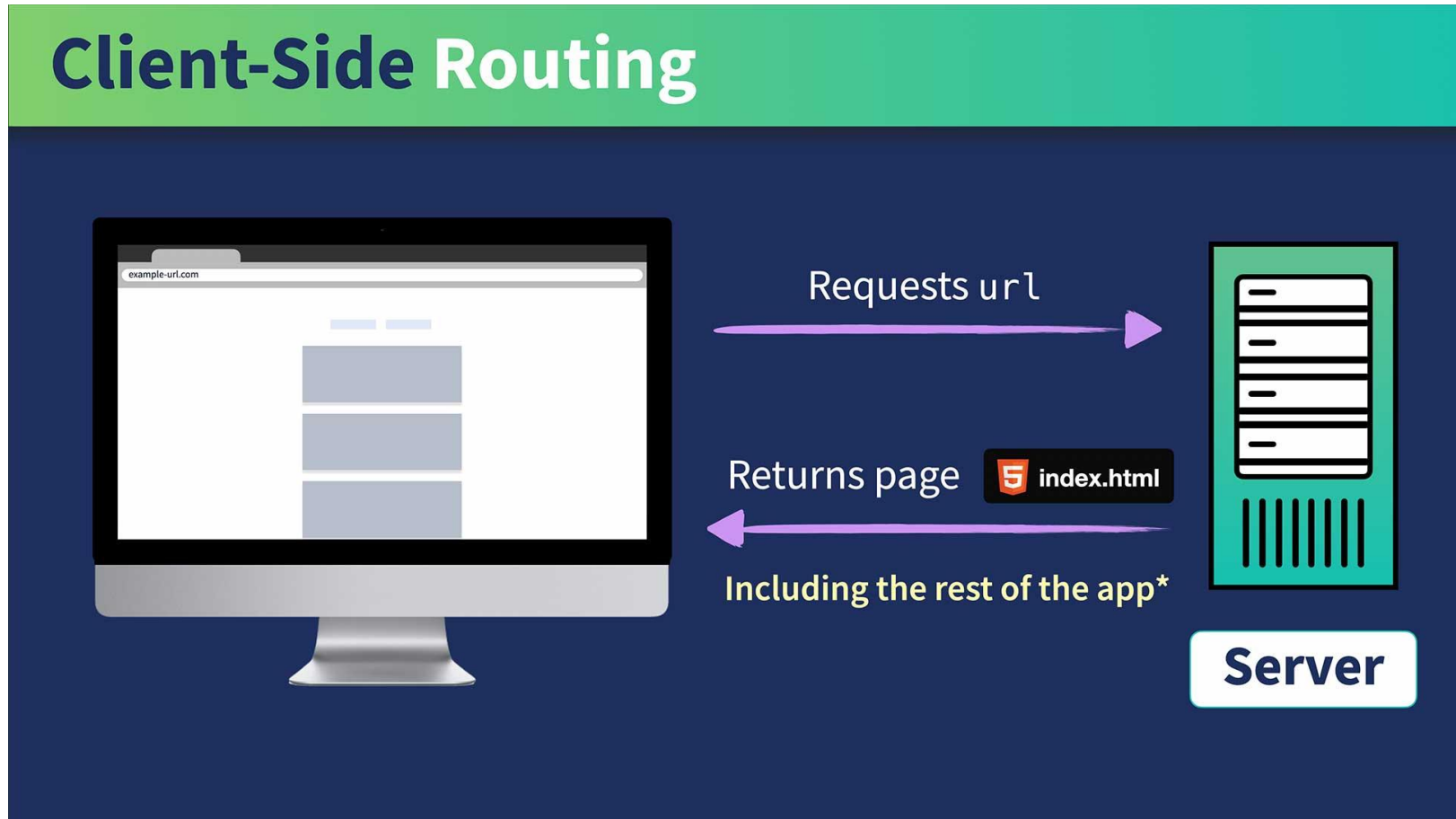
- The target of the Client-site routing
- As the server-side routing
- All the page content are sent via url



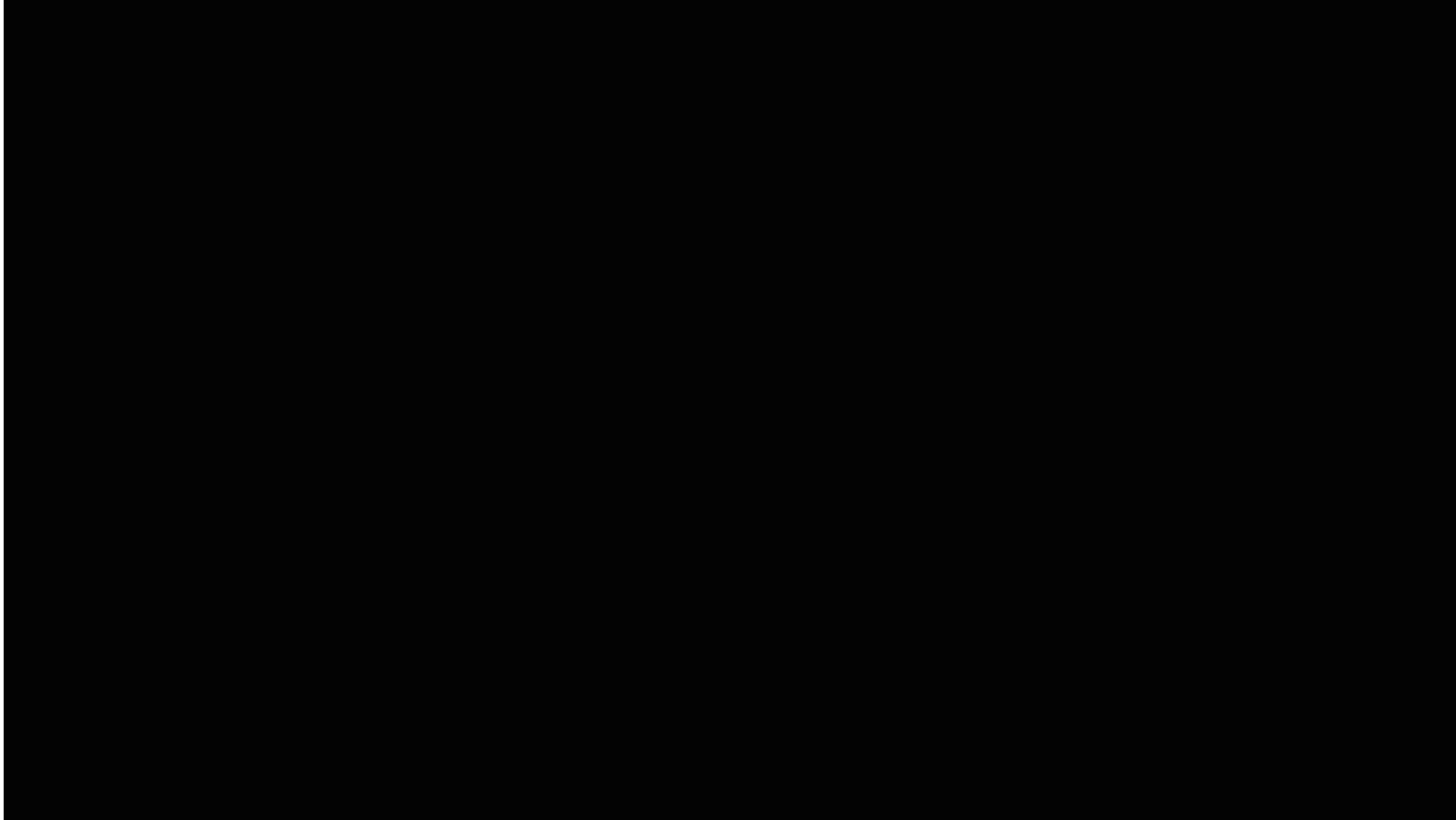
Server-side Routing



Client-Side Routing



Client-Side Routing



To set the Vue-router

- Select when create the app
- The required components are stated in package.json

```
"dependencies": {  
  "pinia": "^2.1.7",  
  "vue": "^3.4.21",  
  "vue-router": "^4.3.0"  
},
```

Vue-router configuration

- In src/router.index.js
- Importing the router component
- Import a component which will route to
- Define the route

```
import { createRouter, createWebHistory } from 'vue-router'  
import HomeView from '@views/HomeView.vue'
```

```
const router = createRouter({  
  history: createWebHistory(import.meta.env.BASE_URL),  
  routes: [  
    {  
      path: '/',  
      name: 'home',  
      component: HomeView  
    },  
  ],  
})
```

- Define the router object, and set the History browser mode

```
export default router
```

Router-link

- Similar to a tag
- But when its click, the component will be replaced in the <router-view/>
- Single page replace the tag in DOM

```
<template>
  <div id="layout">
    <header>
      <div class="wrapper">
        <nav>
          <RouterLink to="/">Home</RouterLink> |
          <RouterLink to="/about">About</RouterLink>
        </nav>
      </div>
    </header>
    <RouterView />
  </div>
</template>
```

```
import { createRouter, createWebHistory } from 'vue-router'
import HomeView from '@views/HomeView.vue'

const router = createRouter({
  history: createWebHistory(import.meta.env.BASE_URL),
  routes: [
    {
      path: '/',
      name: 'home',
      component: HomeView
    },
  ],
})
```

To Get the Data From the backend

- Component for calling the data is required
- The promise technique is needed
 - As the script setup will run when the component is created
 - We can do other thing when we wait for loading data
- Try Task 6

```
interface EventResponse {  
  | data: Event[]  
}  
const events = ref<Event[]>([])  
eventService.getEvents().then((response:EventResponse) => {  
  | events.value = response.data  
}))
```

Using the Axios to call the server

- Promise object is back
- Then
- And catch
- With Type define

```
src > TS type.ts > ...  
1  export interface Event {  
2      id: number  
3      category: string  
4      title: string  
5      description: string  
6      location: string  
7      date: string  
8      time: string  
9      petsAllowed: boolean  
10     organizer: string  
11 }
```

Problem?

- Now the service, and the view is in the same component
- Service
 - How to get the data
- Solution
 - Separate the service to other component

Separation of Concern

- Separate the complicated business logic
- Logic can be reused
- Make the code easy to understand
 - Simplify the.Vue file
- Simplify
 - By removing unrelated import
 - The more import the more complexity of your code

Services

- The folders to keep all services function



- The name of the file is based on the returned entity
- The apiClient
 - The axios object
 - Contains the base url
 - Help to change the base url later

```
import axios from 'axios'
const apiClient = axios.create({
  baseURL: 'http://localhost:3000',
  withCredentials: false,
  headers: {
    Accept: 'application/json',
    'Content-Type': 'application/json',
  },
})

export default {
  getEvents() {
    return apiClient.get('/events')
  },
}
```


Making a call

- The url based will be added automatically

```
export default {  
  getEvents() {  
    return apiClient.get('/events')  
  }  
}
```

```
import axios from 'axios'  
const apiClient = axios.create({  
  baseURL: 'http://localhost:3000',  
  withCredentials: false,  
  headers: {  
    Accept: 'application/json',  
    'Content-Type': 'application/json',  
  },  
})
```

Then replace the code with the new created services

- The promised object is returned
 - then() is used to handle when the data is returned

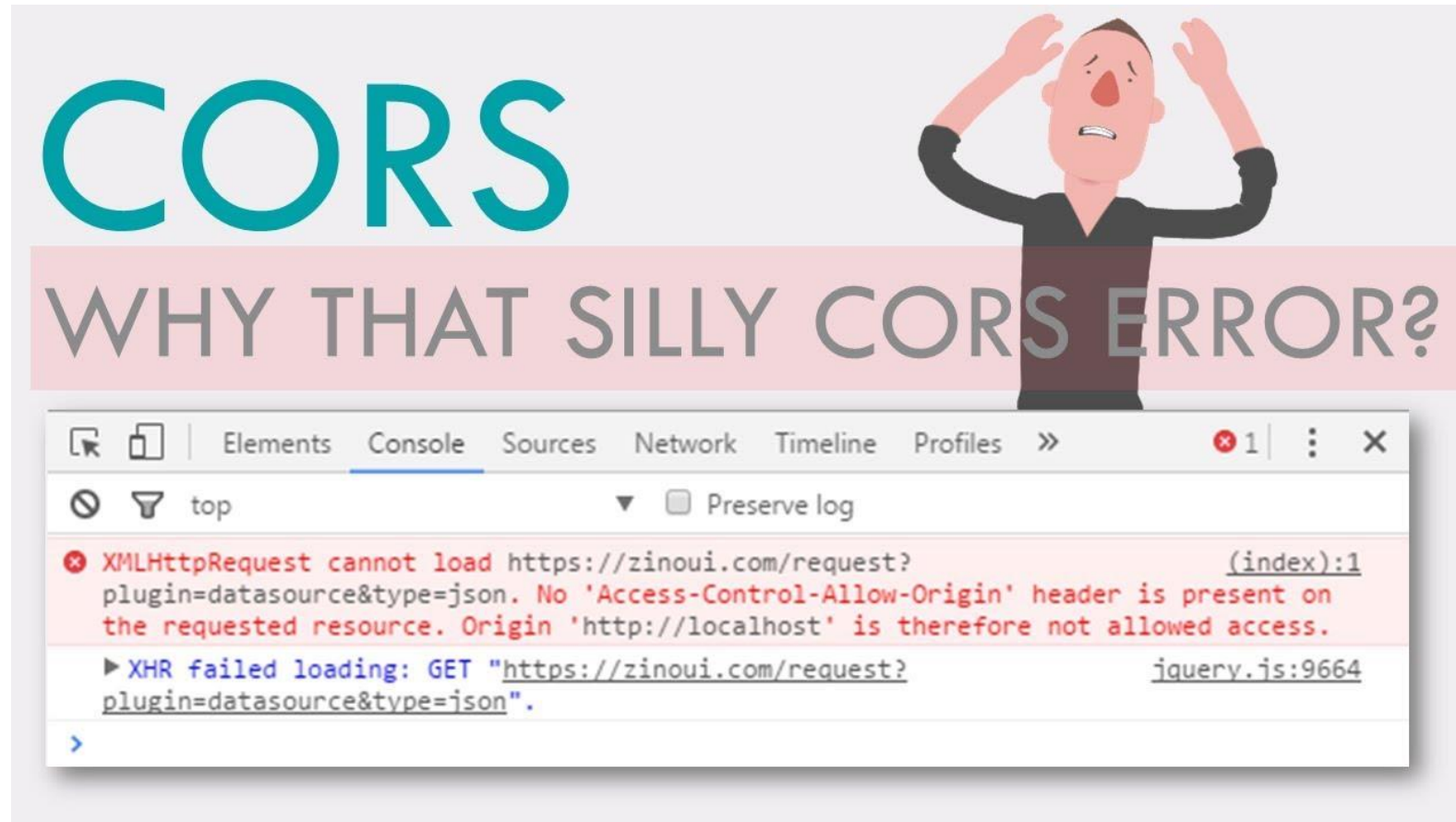
```
interface EventResponse {  
  | data: Event[]  
}  
const events = ref<Event[]>([])  
eventService.getEvents().then((response:EventResponse) => {  
  | events.value = response.data  
})  
</script>
```

The Vue naming convention

- Prefer two words
- To Explain what it is
- Avoid the duplicate name
 - May make confuse
- Example
 - EventListView
 - EventService
 - EventCard

Linking the Frontend to the Backend

- Just set the link?



Cross origin problem

- a mechanism that uses additional [HTTP](#) headers
- to tell a browser to let a web application running at one origin (domain) have permission to access selected resources from a server at a different origin.
- A web application makes a **cross-origin HTTP request** when it requests a resource that has a different origin (domain, protocol, and port) than its own origin.

Security problem

- browsers restrict cross-origin HTTP requests initiated from within scripts.
- For example, XMLHttpRequest and the Fetch API follow the same-origin policy.
- This means that a web application using those APIs can only request HTTP resources from the same origin the application was loaded from,
 - unless the response from the other origin includes the right CORS headers.

CORS Setup

- Create a bean in a configuration
 - @SpringBootApplication can be used as configuration
 - A new bean is created

```
const app = express();  
const allowedOrigins = ['http://localhost:5173'];  
  
const options: cors.CorsOptions = {  
  | origin: allowedOrigins  
};  
  
// Then pass these options to cors:  
app.use(cors(options));  
app.use(express.json());  
app.use('/events', eventRoute);
```

Router

- Select the component to show
- Base on the Url

Dynamic Route

- Rendering the page dynamically
- Some data can be sent via the url

Get URL Parameter

- Path variable
- Query variable

Path Variable URL

- The data has been sent through the url path
- The framework is needed to be set
 - Which part of the URL will be parameter
 - <http://localhost/events/123>
 - <https://www.facebook.com/nbamania23/posts/2313450438788862>
- This technique used to get the dynamic resource data

Query Variable URL

- The data is sent via the query variable
 - The variable is set after ? symbol in the URL
 - The variable is set as the key and value by key = value
 - If there are more than one variable, the variable can be concatenated with &
- <http://localhost:5173/event?id=123>
- <https://www.lazada.co.th/catalog/?q=belt&keyori=ss&from=input&spm=a2o4m.home.search.go.1125719cCgqley>
- Mostly use for query value from web site

Reading the Path Variable URL (Static)

- Setting the path to call the views component

Components/EventCard.vue

router/index.ts

```
{
  path: '/event/5928101',
  name: 'event-detail-view',
  component: EventDetailView
},
```

```
<template>
  <RouterLink to="/event/5928101">
    <div class="event-card">
      <h2>{{ event.title }}</h2>
      <span>@{{ event.time }} on {{ event.date }}</span>
    </div>
  </RouterLink>
</template>
```

Views/EventDetailView.vue

```
<template>
  <div>
    <h1>{{ event.title }}</h1>
    <p>{{ event.time }} on {{ event.date }} @ {{ event.location }}</p>
    <p>{{ event.description }}</p>
  </div>
</template>
```

The loading error

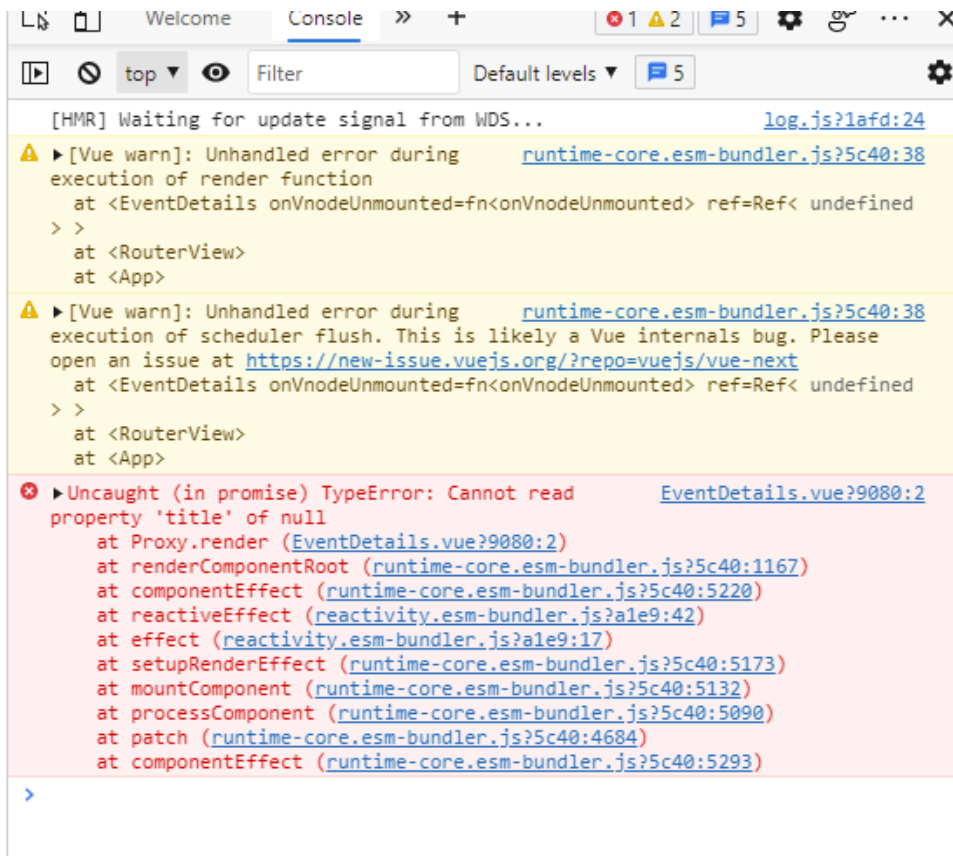
[Home](#) | [About](#)

Cat Adoption Day

12:00 on January 28, 2022 @ Meow Town

Find your new feline friend at this event.

- The Asynchronous call



How To Fix?

```
<template>
  <div v-if="event">
    <h1>{{ event.title }}</h1>
    <p>{{ event.time }} on {{ event.date }} @ {{ event.location }}</p>
    <p>{{ event.description }}</p>
  </div>
</template>
```

- Render the code only when there are event
- Try Task 7

Now the Route should be dynamic

- Using the :id to receive the path variable
- Try task 8

Receiving the id

components/EventCard.vue

```
<RouterLink class="event-link" :to="{ name: 'event-detail-view', params: { id: event.id } }">
  <div class="event-card">
    <h2>{{ event.title }}</h2>
    <span>@{{ event.time }} on {{ event.date }}</span>
  </div>
</RouterLink>
```

Setting the views component

- Setting the props to true, and set the id as path variable

router/index.ts

```
{
  path: '/event/5928101',
  name: 'event-detail-view',
  component: EventDetailView
},
```



```
{
  path: '/event/:id',
  name: 'event-detail-view',
  component: EventDetailView,
  props: true
},
```

```
<template>
  <div v-if="event">
    <h1>{{ event.title }}</h1>
    <p>{{ event.time }} on {{ event.date }} @ {{ event.location }}</p>
    <p>{{ event.description }}</p>
  </div>
</template>
```

views/EventDetailView.vue

```
<script setup lang="ts">
import { ref, onMounted, defineProps } from 'vue'
import type { Event } from '@/types'
import EventService from '@/services/EventService'
const event = ref<Event | null>(null)
const props = defineProps({
  id: {
    type: String,
    required: true
  }
})
onMounted(() => {
  EventService.getEvent(props.id)
    .then((response) => {
      event.value = response.data
    })
    .catch((error) => {
      console.error('There was an error!', error)
    })
})
</script>
```

In run time

```
<template>
  <RouterLink class="event-link" :to="{ name: 'event-detail-view', params: { id: event.id } }">
    <div class="event-card">
      <h2>{{ event.title }}</h2>
      <span>@{{ event.time }} on {{ event.date }}</span>
    </div>
  </RouterLink>
</template>
```

[Home](#) | [About](#)

Events For Good

[@12:00 on January 28, 2022](#)

[Cat Adoption Day](#)

[@10:00 on March 14, 2022](#)

```
<!DOCTYPE html>
<html lang>
  <head>...</head>
  <body>
    <noscript>...</noscript>
    <div id="app" data-v-app>
      <div id="nav">...</div>
      <!-- new element -->
      <h1 data-v-2ad5958e>Events For Good</h1>
      <div class="events" data-v-2ad5958e>
        <a href="/event/123" class data-v-61f707ca data-v-61f707ca>
          <a href="/event/456" class data-v-61f707ca data-v-61f707ca>
            <div class="event-card" data-v-61f707ca>...</div>
          </a>
          <a href="/event/789" class data-v-61f707ca data-v-61f707ca>
            </div>
          </div>
        </div>
      <!-- built files will be auto injected -->
      <script type="text/javascript" src="/js/chunk-vendors.js"></script>
      <script type="text/javascript" src="/js/app.js"></script>
```

In Vue

- Prefer used named routes instead of hardcode path

```
<div class="wrapper">
  <nav>
    <RouterLink to="/">Event</RouterLink> |
    <RouterLink to="/about">About</RouterLink>
  </nav>
```

```
<div class="wrapper">
  <nav>
    <RouterLink :to="{ name: 'event-list-view' }">Event</RouterLink> |
    <RouterLink :to="{ name: 'about' }">About</RouterLink>
  </nav>
</div>
```

Pagination

- There are a lot of data in the world
- All data could not be shown at once
 - Network ability
 - Time consume
 - Server Resources
- Using the pagination
- Open only on the specific page of the data

Pagination

```
{
  "events": [
    {
      "id": 123,
      "category": "animal welfare",
      "title": "Cat Adoption Day",
      "description": "Find your new feline friend at this event.",
    },
    {
      "id": 456,
      "category": "food",
      "title": "Community Gardening",
      "description": "Join us as we tend to the community edible plants.",
    },
    {
      "id": 789,
      "category": "sustainability",
      "title": "Beach Cleanup",
      "description": "Help pick up trash along the shore.",
    },
    {
      "id": 1001,
      "category": "animal welfare",
      "title": "Dog Adoption Day",
      "description": "Find your new canine friend at this event.",
    },
    {
      "id": 1002,
      "category": "food",
      "title": "Canned Food Drive",
      "description": "Bring your canned food to donate to those in need.",
      "location": "Tin City",
    },
    {
      "id": 1003,
      "category": "sustainability",
      "title": "Highway Cleanup",
      "description": "Help pick up trash along the highway.",
    }
  ]
}
```

What we need to know?

- pageSize
- pageNo
- TotalElement

https://my-json-server.typicode.com/chartchai/vue-router/events?_limit=2&_page=1

Name	×	Headers	Payload	Preview	Response	Initiator	Timing	Cookies
{ events?_limit=2&_page=1								
hook-exec.js		▼ Response Headers						
detector-exec.js								
		Access-Control-Allow-Credentials:	true					
		Access-Control-Expose-Headers:	X-Total-Count, Link					
		Alt-Svc:	h3=":443"; ma=86400					
		Cache-Control:	no-cache					
		Cf-Cache-Status:	DYNAMIC					
		Cf-Ray:	8a2cbe88395da072-SIN					
		Content-Encoding:	br					
		Content-Type:	application/json; charset=utf-8					
		Date:	Sat, 13 Jul 2024 22:35:07 GMT					
		Vary:	Origin, Accept-Encoding					
		Via:	1.1 vegur					
		X-Content-Type-Options:	nosniff					
		X-Powered-By:	Express					
		X-Total-Count:	6					

Try Task 9

So, the service is updated

- Update the EventService.js

```
getEvents(perPage: Number, page: Number) {  
  | return apiClient.get('/events?_limit=' + perPage + '&_page=' + page)  
  },  
}
```

What URL will be used?

- In vue, using the query parameter
 - Query data/ only what we need

views/EventListView.vue

```
const events = ref<Event[]>([])
interface Props {
  page: number
}
const props = defineProps<Props>()
const page = computed(() => props.page)

eventService.getEvents(page.value, 2).then((response) => {
  events.value = response.data
})
</script>
```

services/EventService.js

```
getEvents(pageNo: number, pageSize: number) {
  return apiClient.get('/events?pageNo=' + pageNo + '&pageSize=' + pageSize)
},
```

In Routing

router/index.ts

```
{  
  path: '/',  
  name: 'event-list-view',  
  component: EventListView,  
  props: (route) => ({  
    page: parseInt(route.query.page as string) || 1,  
  }),  
},
```

The request URL

<http://localhost:5173/?page=2>

The Prev Menu does not show in the first page?

```
<EventCard v-for="event in events" :key="event.id" :event="event" />
<div class="pagination">
  <RouterLink
    id="page-prev"
    :to="{ name: 'event-list-view', query: { page: page - 1 } }"
    rel="prev"
    v-if="page != 1"
    >Prev Page</RouterLink
  >
</div>
```

What Happen when click Next?

- The page is not load!!!!
- The load is created only once
 - The created is call only one time

```
const props = defineProps<Props>()
const page = computed(() => props.page)

eventService.getEvents(page.value, 2).then((response) => {
  | events.value = response.data
})
</script>
```

Why the page in url is changed?

```
<RouterLink  
  id="page-next"  
  :to="{ name: 'event-list-view', query: { page: page + 1 } }"  
  rel="next"  
>Next Page</RouterLink  
>
```

```
const props = defineProps<Props>()  
const page = computed(() => props.page)  
  
eventService.getEvents(page.value, 2).then((response) => {  
  | events.value = response.data  
  })  
</script>
```

- Query page is changed computed value help change the page

What is Computed Property

- Data is the static data
 - Provided at the beginning of the component creation
- Data need to be computed
 - If compute only, use the method
- Compute and want to show
 - Use computed property

Computed Property

- Call the computed function, which receive the functions which will executed when the data has been change (Functional Programming)

```
const title = computed(() =>{  
  return brand.value + ' ' + product.value  
})
```

- Input function should return values
- The computed function should be loaded

```
const { createApp, ref, computed } = Vue
```

- bind as simple property

```
<div class="product-info">  
  <h1>{{ title}}</h1>
```


Solution

- Use watchEffect()
 - Use for watching the data change
 - If data changed, the function will be called again
 - First publish in Vue 3.0

```
import { ref, onMounted, computed, watchEffect } from 'vue'
```

```
watchEffect(() => {  
  eventService  
    .getEvents(page.value, 2)  
    .then((response) => {  
      events.value = response.data  
    })  
    .catch((error) => {  
      console.error('There was an error!', error)  
    })  
})
```

Calculating the TotalPage

- Reading the header data

```
const events = ref<Event[]>([])
const totalEvents = ref(0)
watchEffect(() => {
  eventService
    .getEvents(page.value, 2)
    .then((response) => {
      events.value = response.data
      totalEvents.value = response.headers['x-total-count']
    })
})
```

Defined LastPage

- Next button should not shown at the last page

```
const hasNextPage = computed(() => {  
  const totalPages = Math.ceil(totalEvents.value / 2)  
  return page.value < totalPages  
})
```

```
<RouterLink  
  id="page-next"  
  :to="{ name: 'event-list-view', query: { page: page + 1 } }"  
  rel="next"  
  v-if="hasNextPage"  
>Next Page</RouterLink  
>
```

Not Working!!!

- Backend does not allow some headers to be returned
- Update the backend

```
res.setHeader("x-total-count", result.count.toString());  
res.setHeader("Access-Control-Expose-Headers", "x-total-count");  
res.json(result.events);
```

Resource Actions

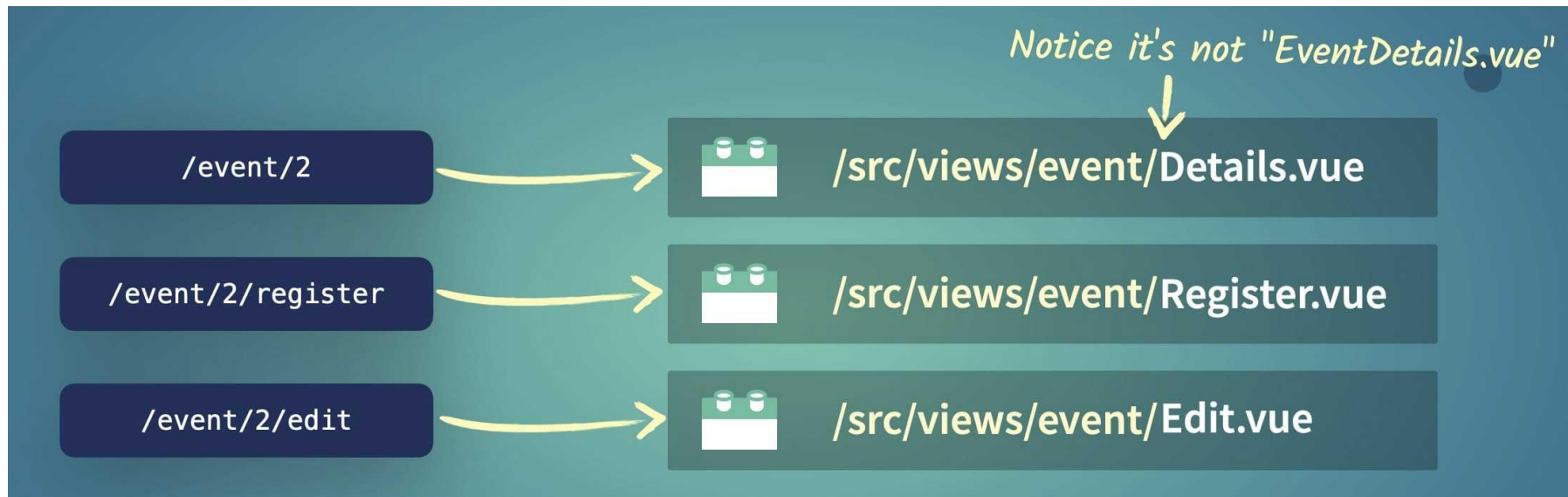
- With a single resource (Event)
 - We want to do different action
 - View, edit, register
- URL must be provided for each action
- What should it be?

Sample urls

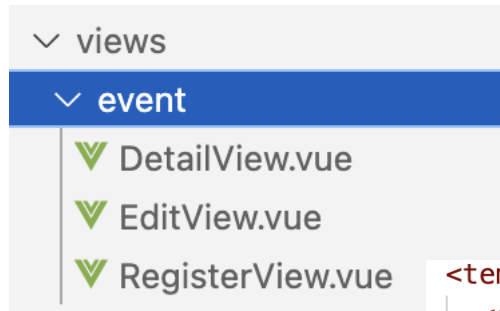
- `/event/2` - Event Details (information)
- `/event/2/register` - Event Register (to signup for the event)
- `/event/2/edit` - Event Edit (to edit the event)
- Where should we place the file?
 - Nested is required
 - If everything in the same views folder will be messy when we have more resources

File Layout

- The best practice is create the own Event (resource) folder
- Try Task 10



The Layout



Menu is added in all file

```
<template>
  <div v-if="event">
    <h1>{{ event.title }}</h1>
    <nav>
      <router-link :to="{ name: 'event-detail-view', params: { id } }">Details</router-link>
      |
      <router-link :to="{ name: 'event-register-view', params: { id } }">Register</router-link>
      |
      <router-link :to="{ name: 'event-edit-view', params: { id } }">Edit</router-link>
    </nav>
    <p>{{ event.time }} on {{ event.date }} @ {{ event.location }}</p>
    <p>{{ event.description }}</p>
  </div>
</template>
```

```
{
  path: '/event/:id',
  name: 'event-detail-view',
  component: EventDetailView,
  props: true
},
{
  path: '/event/:id/register',
  name: 'event-register-view',
  component: EventRegisterView,
  props: true
},
{
  path: '/event/:id/edit',
  name: 'event-edit-view',
  component: EventEditView,
  props: true
},
}
```


Problem?

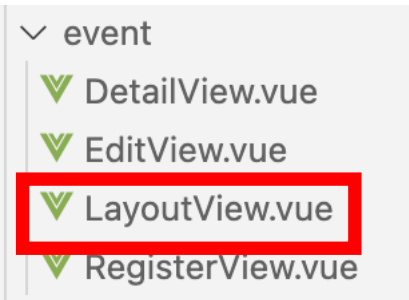
- Duplicate code
- Hard to update if required

Nested Route

- Route under the Route
- Share same resource

New File Layout

event/LayoutView.vue



```
onMounted(() => {
  EventService.getEvent(parseInt(props.id))
    .then((response) => {
      event.value = response.data
    })
    .catch((error) => {
      console.error('There was an error!', error)
    })
})
</script>
<template>
  <div v-if="event">
    <h1>{{ event.title }}</h1>
    <nav>
      <RouterLink :to="{ name: 'event-detail-view', params: { id } }">Details</RouterLink>
      |
      <RouterLink :to="{ name: 'event-register-view', params: { id } }">Register</RouterLink>
      |
      <RouterLink :to="{ name: 'event-edit-view', params: { id } }">Edit</RouterLink>
    </nav>
    <RouterView :event="event" />
  </div>
```

Children views

```
const props = defineProps<{  
  event: Event  
const { event } = toRefs(props)  
</script>
```

```
<template>
```

```
<p>{{ event.time }} on {{ event.date }} @ {{ event.location }}</p>
```

```
<p>{{ event.description }}</p>
```

```
</template>
```

```
</template>
```

```
</template>
```

event/DetailView.vue

event/EditView.vue

```
const { event } = toRefs(props)
```

```
</script>
```

```
<template>
```

```
<p>Edit event here</p>
```

```
</template>
```

event/RegisterView.vue

```
// eslint-disable-next-line @typescript-eslint/no-unused-vars
```

```
const { event } = toRefs(props)
```

```
</script>
```

```
<template>
```

```
<p>Register event here</p>
```

```
</template>
```

Link View

- Using Nested Route

```
{
  path: '/event/:id',
  name: 'event-layout-view',
  component: EventLayoutView,
  props: true,
  children: [
    {
      path: '',
      name: 'event-detail-view',
      component: EventDetailView,
      props: true
    },
    {
      path: 'register',
      name: 'event-register-view',
      component: EventRegisterView,
      props: true
    },
    {
      path: 'edit',
      name: 'event-edit-view',
      component: EventEditView,
      props: true
    }
  ]
}
```

Pragmatic Navigation

- Previously
 - View changed by the action of the user

```
<RouterLink class="event-link" :to="{ name: 'event-detail-view', params: { id: event.id } }">
  <div class="event-card">
    <h2>{{ event.title }}</h2>
    <span>@{{ event.time }} on {{ event.date }}</span>
  </div>
</RouterLink>
```

- But if when we click some button
 - we may need some calculation before moving to the other view
 - Writing the program to drive the view
- Try Task 11

The mapping Event

- Data Binding
 - Bind the action from the user
 - To action in the source code
 - Aka the functions

Event handling

- Event that can be used in the HTML
- [JavaScript Events \(w3schools.com\)](https://www.w3schools.com/js/js_events.asp)
https://www.w3schools.com/js/js_events.asp
- We can bind the script using v-on

```
<button class="button" v-on:click="addToCart">Add to Cart</button>
```


v-on

- Similar to v-bind
 - v-on: -> event name after semicolon
 - Should be assigned to the script method

```
, ---  
<button class="button" v-on:click="addToCart">Add To Cart</button>
```

```
function addToCart() {  
  cart.value +=1  
}  
return {  
  product,  
  image,  
  inStock,  
  inventory,  
  details,  
  variants,  
  cart,  
  addToCart  
}
```

```
const cart = ref(0)
```

The function

- Can be declared in any location of the setup method
- Best practice for composition API
 - Group the variable and function which related to each other together
- Note that to get or change the value of ref object
 - The .value is used

```
function addToCart() {  
  |   cart.value +=1  
}
```

The shorthand

- v-on make the code a bit long
- Shorthand -> @event_name

```
<button class="button" v-on:click="addToCart">Add to Cart</button>
```

```
<button class="button" @click="addToCart">Add to Cart</button>
```

Pracmatic Navigation

event/DetailView.vue

```
import { useRouter } from 'vue-router'
```

```
const router = useRouter()  
const register = () => {  
  router.push({ name: 'event-detail-view' })  
}
```

```
children: [  
  {  
    path: '',  
    name: 'event-detail-view',  
    component: EventDetailView,  
    props: true  
  },  
]
```

The other router paramters

- When you click on a `<router-link>` it's simply calling the `router.push` function inside the code
 - The options are the same as `<router-link>`

```
// Directly to the path with a single string
router.push('/about')

// Directly to the path with an object
router.push({ path: '/about' })

// Directly to the named path
router.push({ name: 'About' })

// With a dynamic segment as I showed above
router.push({ name: 'EventDetails', params: { id: 3 } })

// With a query ... resulting in /?page=2 in our example app
router.push({ name: 'EventList', query: { page: 2 } })
```

Go Back and Go Forward

- Instead of pushing back button or forward in the browser
- Or it is hard to push the back button or forward button in the mobile app.
- We can provide the pragmatic navigation using `go` function

```
// Go forward a record  
router.go(1)  
  
// Go backward a record  
router.go(-1)
```

Error Handling

- Error
 - The thing that we do not want it to be happened
 - But it may happen
- Handling
 - Handle the error
 - For better UX
- Try Task 12

The input URL is not found

```
{  
  path: '/*:catchAll(.*)',  
  name: 'not-found',  
  component: NotFoundView  
}
```

- catchAll – catch other url
 - The url which is not defined earlier
- When type anything
 - The NotFoundView will be shown

The input URL is not found

```
{  
  path: '/*',  
  name: 'not-found',  
  component: NotFoundView    views/NotFoundView.vue  
}
```

```
<template>  
  <h1>Oops!</h1>  
  <h3>The page you're looking for is not here.</h3>  
  <router-link :to="{ name: 'event-list-view' }">Back to the home page</router-link>  
</template>
```

Dynamic Error

Error when making a request

views/NotFoundView.vue

- If the return code is not 200
 - The error will be thrown

views/event/LayoutView.vue

```
const router = useRouter()

onMounted(() => {
  EventService.getEvent(parseInt(props.id))
    .then((response) => {
      event.value = response.data
    })
    .catch(() => {
      router.push({
        name: '404-resource-view',
        params: { resource: 'event' }
      })
    })
})
```

router/index.js

```
{
  path: '/404/:resource',
  name: '404-resource-view',
  component: NotFoundView,
  props: true
},
```

```
import { defineProps, withDefaults } from 'vue' 556.5k (gzipped: 178.4k)
const props = withDefaults(
  defineProps({
    resource: string
  })(),
  {
    resource: 'page'
  }
)
</script>
<template>
  <h1>Oops!</h1>
  <h3>The {{ resource }} you're looking for is not here.</h3>
  <router-link :to="{ name: 'event-list-view' }">Back to the home page</router-link>
</template>
```

- router -> the router of this component
 - push -> = click <router-link :to="name:'404Resource',params: resource:'event'">

Checking other error code

- 404 is the resource not Found
- There may be other code as well
- [HTTP response status codes - HTTP | MDN \(mozilla.org\)](https://developer.mozilla.org/en-US/docs/Web/HTTP/Status)
(<https://developer.mozilla.org/en-US/docs/Web/HTTP/Status>)
- Other error

Check for other errors

views/event/LayoutView.vue

```
onMounted(() => {
  EventService.getEvent(parseInt(props.id))
    .then((response) => {
      event.value = response.data
    })
    .catch((error) => {
      if (error.response && error.response.status === 404) {
        router.push({
          name: '404-resource-view',
          params: { resource: 'event' }
        })
      } else {
        router.push({ name: 'network-error-view' })
      }
    })
})
```

router/index.js

```
{
  path: '/network-error',
  name: 'network-error-view',
  component: NetworkErrorView
},
```

views/NetworkErrorView.vue

```
<template>
  <div class="networkError">
    <h1>Uh-Oh!</h1>

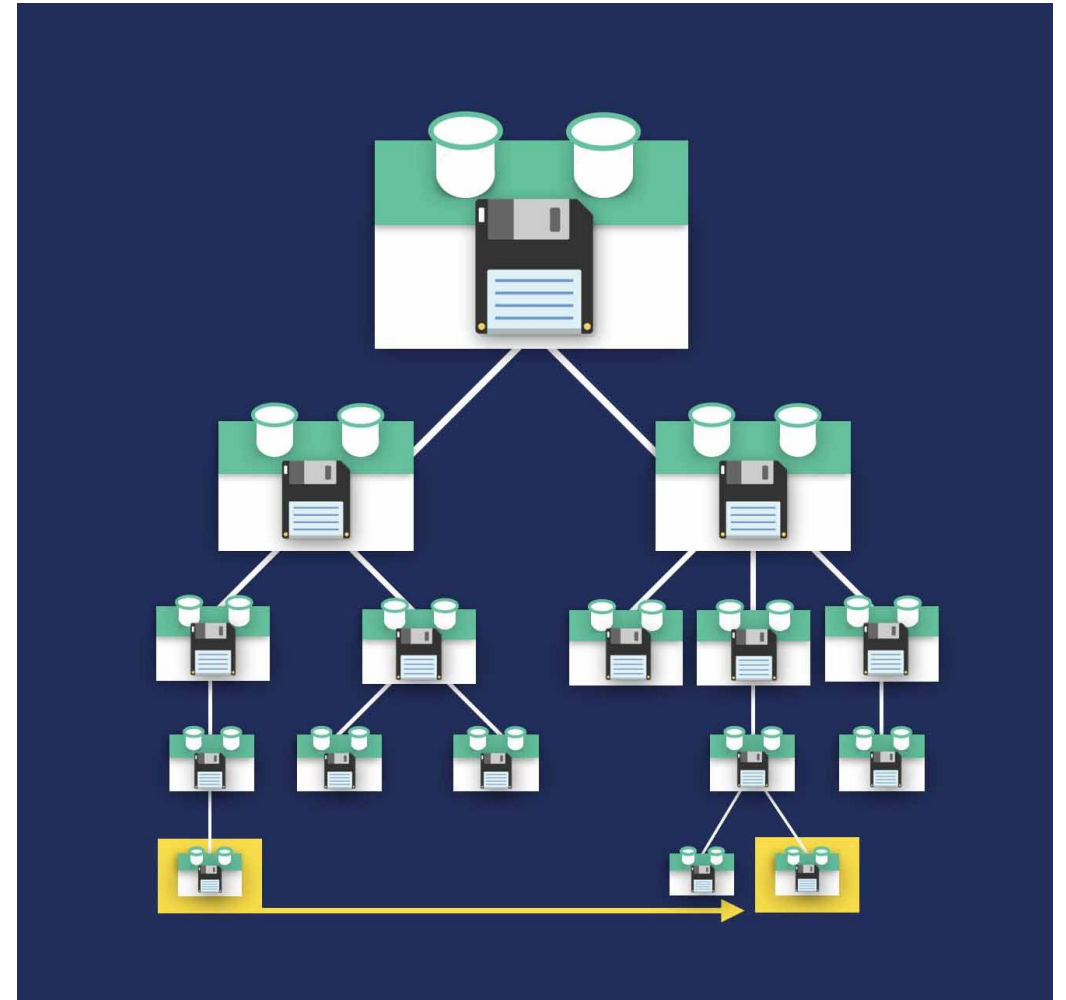
    <h3>
      It looks like you're experiencing some network issues, please take a breath and
      <a href="#" @click="$router.go(-1)">click here</a> to try again.
    </h3>
  </div>
</template>
```

State Management

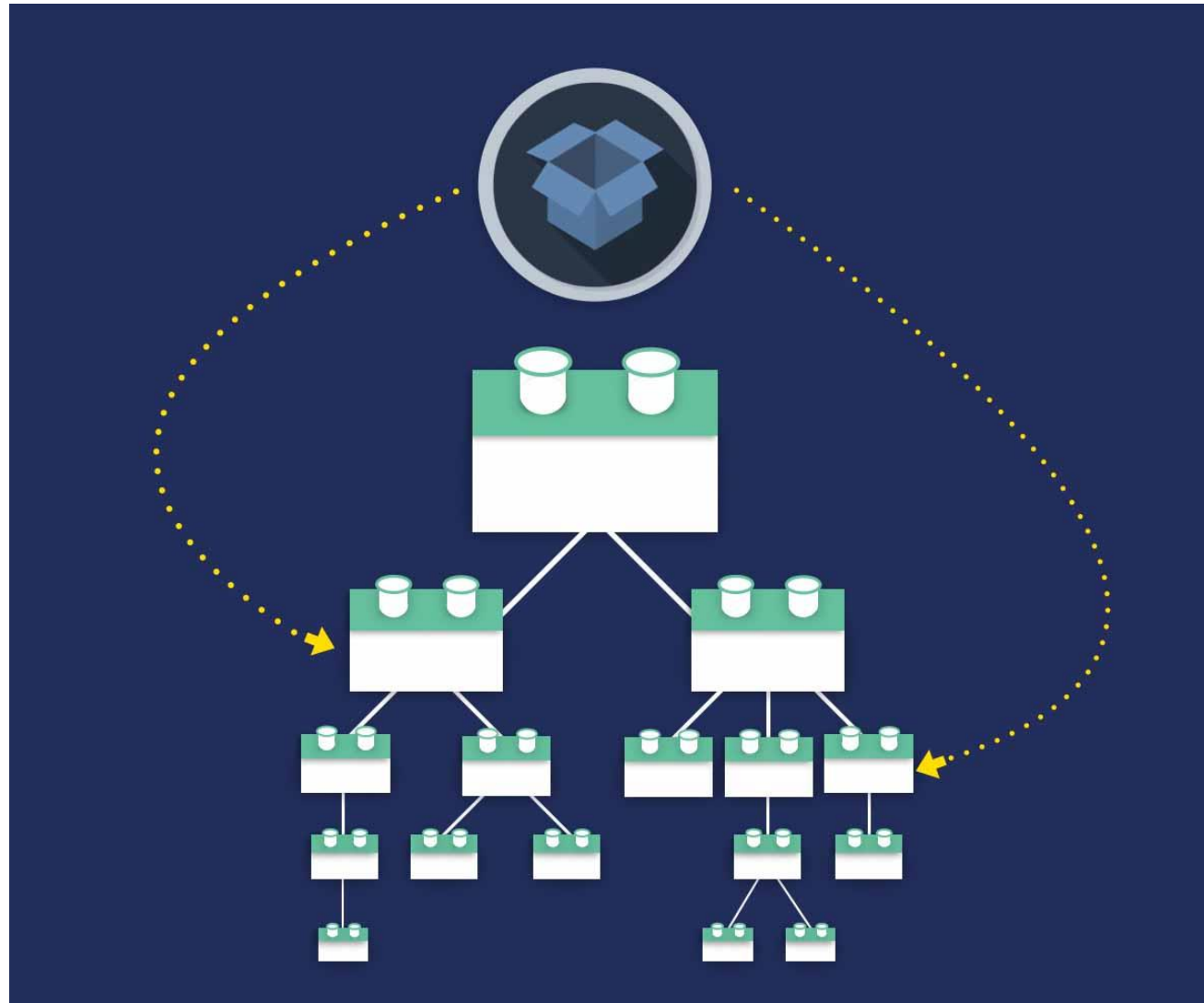
- Manage the Single Source of Truth
- Holding the state
- Manage the changing the state

State?

- In an application, some things you see on the screen don't change.
 - But other things have to change based on certain situations.
- For example, login
 - The status of the other page should be changed regarding to the login result



Using emit, props?

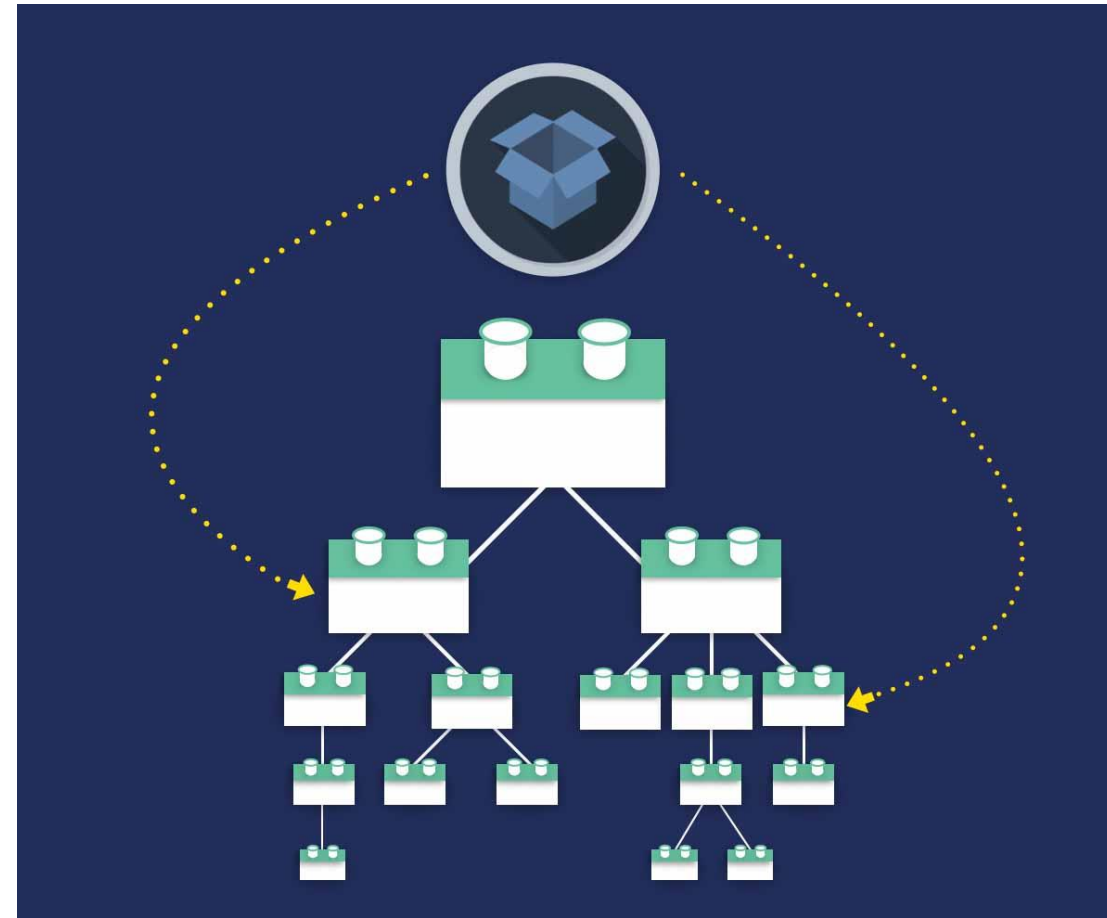


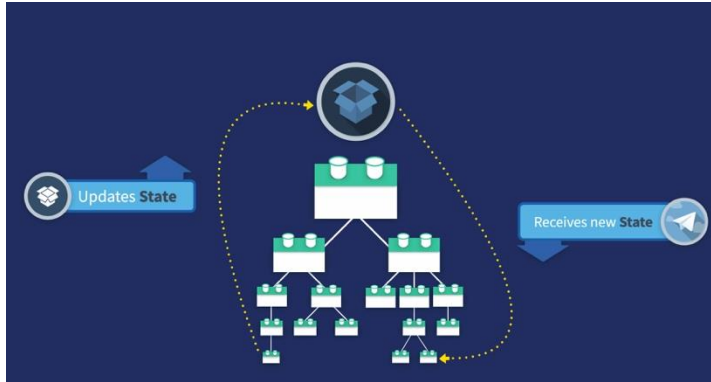
Problems

- Not convenient
- Messy and hard to manage

Why State management?

- Not good for simple global state
- Using state management
 - Allow only some operation
 - Clean up the value before store in state





A mental model of Pinia

- Complete store with a simple built-in system

```
Pinia Store ({
```

```
  state: {  
    ...  
  },
```

```
  actions: {  
    ...  
  },
```

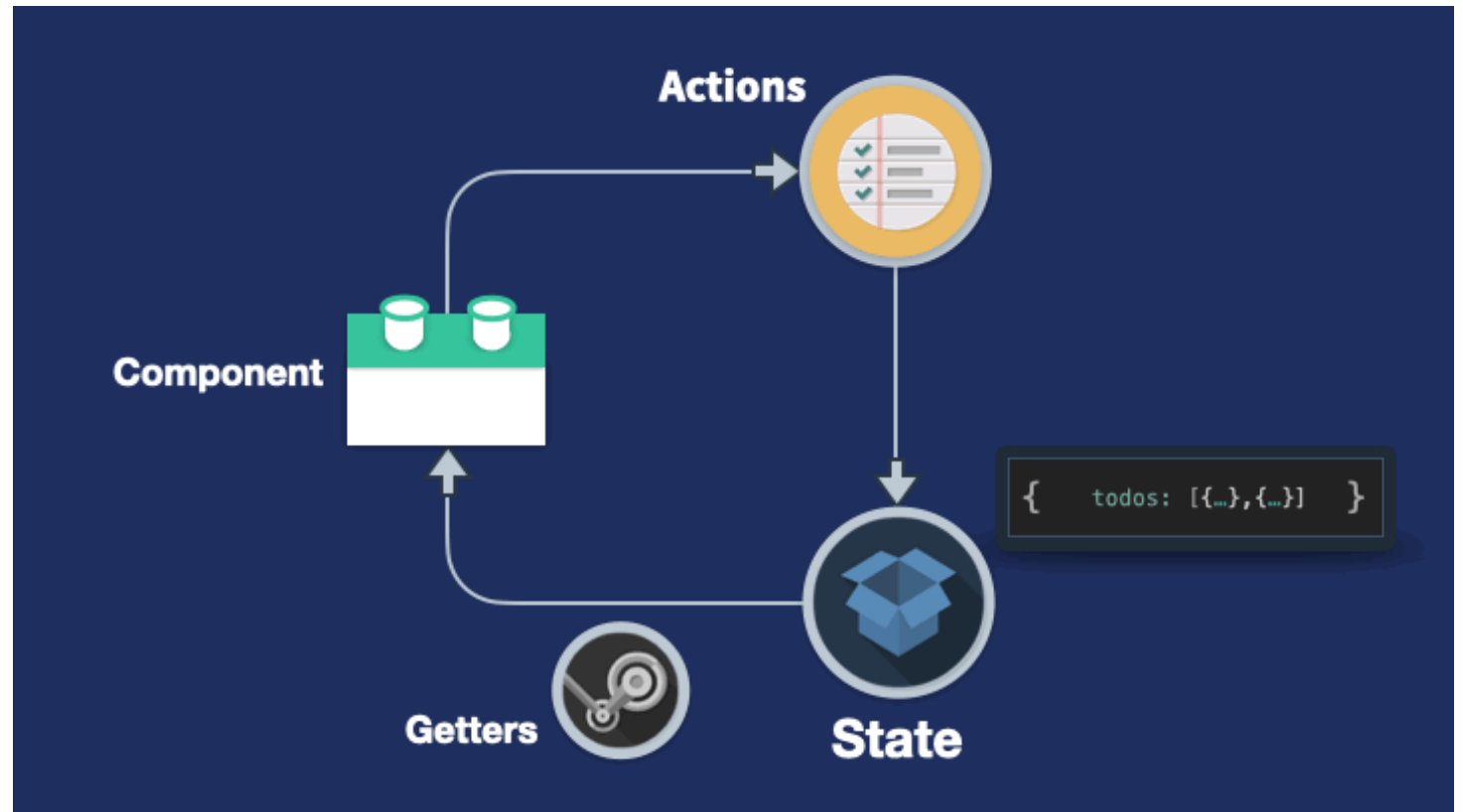
```
  getters: {  
    ...  
  }  
}
```

```
})
```



Pinia in motion

- action
 - The methods to update the state of the store
- getter
 - Return the processed version of the state



Example

```
import { defineStore } from 'pinia'
export const useTodoStore = defineStore('ToDoStore', {
  state: () => ({
    todos: []
  }),
  actions: {
    addTodo(todo){
      this.todos.push(todo)
    }
  },
  getters: {
    doneTodos: (state) => state.todos.filter(todo => todo.done)
  }
})
```

Try Task 13

Define the Type

- Each store contains the type
 - The type can be defined
 - As the composite type
- Type provides values which we want to keep in the store

```
export interface MessageState {  
  | message: string  
}
```

Defining the Store

- defineStore function
- The function which return the MessageState type
 - Without define here
 - The type will be any

```
import type { MessageState } from '@types'
import { defineStore } from 'pinia' 4.7k (gzipped: 2.2k)
export const useMessageStore = defineStore('message', {
  state: (): MessageState => ({
    message: ''
  }),
  actions: {
    updateMessage(message: string): void {
      this.message = message
    },
    resetMessage(): void {
      this.message = ''
    }
  }
})
```



state

- Provide the state object
 - The initial value

```
import type { MessageState } from '@types'
import { defineStore } from 'pinia' 4.7k (gzipped: 2.2k)
export const useMessageStore = defineStore('message', {
  state: (): MessageState => ({
    message: ''
  }),
  actions: {
    updateMessage(message: string): void {
      this.message = message
    },
    resetMessage(): void {
      this.message = ''
    }
  }
})
```

action

- What we can do with the state
- Define as function
 - Return type should be provided
- Mostly to update the value of the state

```
import type { MessageState } from '@types'
import { defineStore } from 'pinia' 4.7k (gzipped: 2.2k)
export const useMessageStore = defineStore('message', {
  state: (): MessageState => ({
    message: ''
  }),
  actions: {
    updateMessage(message: string): void {
      this.message = message
    },
    resetMessage(): void {
      this.message = ''
    }
  }
})
```

Using the Store

- Import the const function
- Create the local variable

message.ts

```
import { defineStore } from 'pinia'
export const useMessageStore = defineStore('message', {
  state: () => ({
    message: '' as string
  }),
  actions: {
    updateMessage(message: string) {
      this.message = message
    },
    resetMessage() {
      this.message = ''
    }
  }
})
```

event/RegisterView.vue

```
import { useMessageStore } from '@stores/message'
const store = useMessageStore()
const register = () => {
  store.updateMessage('You are successfully registered for ' + props.event.title)
  setTimeout(() => {
    store.resetMessage()
  }, 3000)
  router.push({ name: 'event-detail-view', params: { id: props.event.id } })
}
```

App.vue

```
import { useMessageStore } from '@stores/message'
import { storeToRefs } from 'pinia' 4.2k (gzipped: 2k)
const store = useMessageStore()
const { message } = storeToRefs(store)
```

```
<header>
  <div id="flashMessage" v-if="message">
    <h4>{{ message }}</h4>
  </div>
```

StoreToRefs

- Deconstruction the value in the state
- Use as the reactive object in single file components
- Allows to update the data in the components

```
-----\nconst { message } = storeToRefs(store)
```

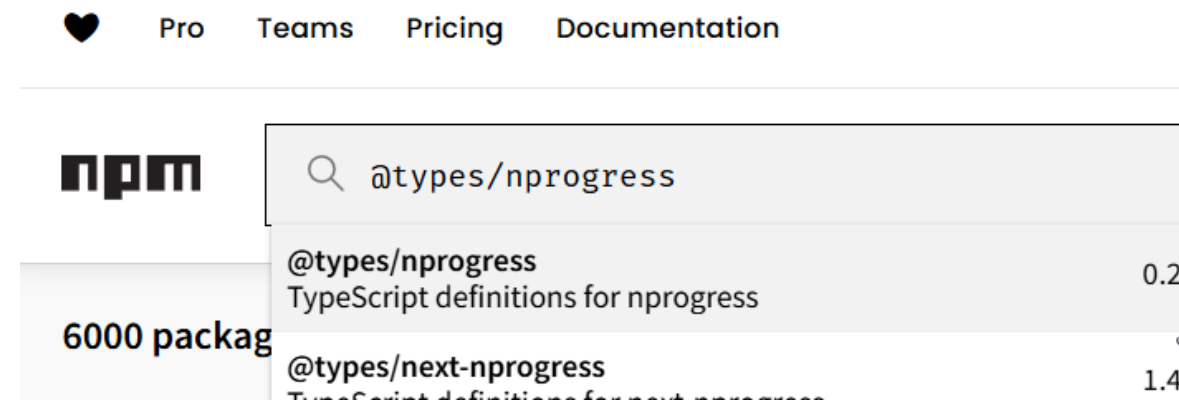
```
import { defineStore } from 'pinia'\nexport const useMessageStore = defineStore('message', {\n  state: () => ({\n    message: '' as string\n  }),\n  actions: {\n    updateMessage(message: string) {\n      this.message = message\n    },\n    resetMessage() {\n      this.message = ''\n    }\n  }\n})
```

Using the Component

- NProgress
 - Reference [nprogress - npm \(npmjs.com\)](https://www.npmjs.com/package/nprogress)
 - Provide by rstacruz
 - Use for creating the progressbar
- Required to make the better UX for the user
- Try Task 14

Using the component with typescript

- Working with type script?
 - The type is required
- Some components has implemented in pure java
- If we want to use the type
 - Some components provide the types of component
 - Now the component is supporting the type
 - Autocomplete
 - Code suggestion (field, functions)
- If there is no type
 - The any type is applied
 - No code suggestion
 - Follow the docs



Try-catch-finally

- Review

- try

- Execute on anything if the error occurs jump from the code to the error section
 - The Error object must be throws

- catch

- Receive the error object and execute

- finally

- After finish try block or catch box
 - The finally will always call

```
onMounted(() => {  
  EventService.getEvent(parseInt(props.id))  
    .then((response) => {  
      event.value = response.data  
    })  
    .catch((error) => {  
      if (error.response && error.response.status === 404) {  
        router.push({  
          name: '404-resource-view',  
          params: { resource: 'event' }  
        })  
      } else {  
        router.push({ name: 'network-error-view' })  
      }  
    })  
})  
})
```

Router Hook

- The life cycle hook
- But for the router action
- Called when the Router event is created

Global Route Guard

- The guard which apply to all route
 - `beforeEach((to,from,next)=>{})`
 - Before all the route
 - `beforeResolve ((to,from,next)=>{})`
 - After all in-component guards and async route component are resolved
 - `afterEach((to,from)=>{})`
 - Call when the hook is finished
 - No `next()` function as the route transition is finish

The return value for the Navigation Guard

- The return value is used instead
- No return value – will continue navigation
- return true – will continue navigation
- return false – cancel the navigation
- return '/' – redirect page to the / path
- return {name: 'event-list'} – redirects to the name path

```
data: function() {
  return {
    unsavedChanges: false // <-- Flag gets set to true if anything
                          // is changed on the form
  }
},
beforeRouteLeave(routeTo, routeFrom) {
  if (this.unsavedChanges) {
    const answer = window.confirm(
      'Do you really want to leave? You have unsaved changes!'
    )
    if (!answer) {
      return false // <-- Cancels the navigation
    }
  }
}
```

With Router version4

- The next can be remove
- The return value is used instead
- No return value – will continue navigation
- return true – will continue navigation
- return false – cancel the navigation
- return '/' – redirect page to the / path
- return {name: 'evet-list'} – redirects to the name path

```
data: function() {  
  return {  
    unsavedChanges: false // <-- Flag gets set to true if anything  
                           // is changed on the form  
  }  
},  
beforeRouteLeave(routeTo, routeFrom) {  
  if (this.unsavedChanges) {  
    const answer = window.confirm(  
      'Do you really want to leave? You have unsaved changes!'  
    )  
    if (!answer) {  
      return false // <-- Cancels the navigation  
    }  
  }  
}
```

With the Global Route Guard

- The same activity apply for all the route
- Benefit
 - Remove Redundancy
 - Easy to maintain
 - Easy to upgrade

Setting the Global Navigation Guard

```
const router = createRouter({  
  history: createWebHistory(import.meta.env.BASE_URL),  
  routes: [  
  
    router.beforeEach(() => {  
      | nProgress.start()  
    })  
  
    router.afterEach(() => {  
      | nProgress.done()  
    })  
  ]  
})
```

Per-Route Guards

- Setup the things to be done before each navigation
- Setting the alternative navigation if there are some exceptions
- Remove the preparation/controller in the view to the router management
- Try Task 15

Per-Route Guard

- beforeEnter
- Applied after before each
- Can be a list of function called

```
const routes = [  
  {  
    path: '/users/:id',  
    component: UserDetails,  
    beforeEnter: (to, from) => {  
      // reject the navigation  
      return false  
    },  
  },  
]
```

```
function removeQueryParams(to) {  
  if (Object.keys(to.query).length)  
    return { path: to.path, query: {}, hash: to.hash }  
}  
  
function removeHash(to) {  
  if (to.hash) return { path: to.path, query: to.query, hash: '' }  
}  
  
const routes = [  
  {  
    path: '/users/:id',  
    component: UserDetails,  
    beforeEnter: [removeQueryParams, removeHash],  
  },  
  {  
    path: '/about',  
    component: UserDetails,  
    beforeEnter: [removeQueryParams],  
  },  
]
```

beforeEnter

- Execute before navigate to target folder
- The route can be changed
- Action can be updated
 - Put the data in the store
- The warning for to is shown
 - Lint check that to is not used
 - To fix... later

```
--  
{  
  path: '/event/:id',  
  name: 'event-layout-view',  
  component: EventLayoutView,  
  props: true,  
  beforeEnter: (to) => {  
    | // put API call here  
  },  
}
```


Note

- Per-route guard does not apply for children
- Use `beforeEach` or `beforeEnter` in the child node

Putting the object in the Per-Route

- Provide a new Store (the old store can be used if required)

router/index.ts

types.ts

```
export interface EventState {  
  event: Event | null  
}
```

store/event.ts

```
import type { MessageState } from '@/types'  
import { defineStore } from 'pinia' 4.7k (gzipped: 2.1k)  
export const useMessageStore = defineStore('message', {  
  state: (): MessageState => ({  
    message: ''  
  }),  
  actions: {  
    updateMessage(message: string): void {  
      this.message = message  
    },  
    resetMessage(): void {  
      this.message = ''  
    }  
  }  
})
```

```
{  
  path: '/event/:id',  
  name: 'event-layout-view',  
  component: EventLayoutView,  
  props: true,  
  beforeEnter: (to) => {  
    const id = parseInt(to.params.id as string)  
    const eventStore = useEventStore()  
    return EventService.getEvent(id)  
      .then((response) => {  
        // need to setup the data for the event  
        eventStore.setEvent(response.data)  
      })  
      .catch((error) => {  
        if (error.response && error.response.status === 404) {  
          return {  
            name: '404-resource-view',  
            params: { resource: 'event' }  
          }  
        } else {  
          return { name: 'network-error-view' }  
        }  
      })  
  }  
},  
...  
}
```

Reading the store

- Clearing the usage of props

```
const event = ref<Event | null>(null)
const props = defineProps({
  id: {
    type: String,
    required: true
  }
})
const router = useRouter()

onMounted(() => {
  EventService.getEvent(parseInt(props.id))
    .then((response) => {
      event.value = response.data
    })
    .catch((error) => {
      if (error.response && error.response.status === 404) {
        router.push({
          name: '404-resource-view',
          params: { resource: 'event' }
        })
      } else {
        router.push({ name: 'network-error-view' })
      }
    })
})
</script>
```

```
<script setup lang="ts">
import { useEventStore } from '@/stores/event'
import { storeToRefs } from 'pinia'

const store = useEventStore()
const { event } = storeToRefs(store)

</script>
<template>
  <div v-if="event">
```

Cleaner Code

- But More Dependency
- The Component needs to link to the store
- Consider the use of it

Vue Router

- Provide the component to handle the back button automatically

```
const router = createRouter({  
  history: createWebHistory(import.meta.env.BASE_URL),  
  routes: [  
    // ...  
  ]  
})
```

- Override the default back button click behavior

As it is only one page

- When updating the content
 - The browser location is the same
- Need to set the behavior of browser when updating the component

Setting the position

- Setting the position to the top of the page
 - Easier for the user

```
],  
  scrollBehavior() {  
    return { top: 0 }  
  }  
}
```

- Return can be any position
- Can set to any element (plus location)

```
const router = createRouter({  
  scrollBehavior(to, from, savedPosition) {  
    // always scroll 10px above the element #main  
    return {  
      // could also be  
      // el: document.getElementById('main'),  
      el: '#main',  
      top: -10,  
    }  
  },  
})  
})
```

savedPosition

- When the component change
 - The position of the old position is saved
- Simulate the navigation with back/forward button

```
    ],  
    scrollBehavior(to, from, savedPosition) {  
      if (savedPosition) {  
        return savedPosition  
      } else {  
        return { top: 0 }  
      }  
    }  
  }  
}
```

- More details
 - [Scroll Behavior | Vue Router \(vuejs.org\)](https://next.router.vuejs.org/guide/advanced/scroll-behavior.html)
 - <https://next.router.vuejs.org/guide/advanced/scroll-behavior.html>

- Finally Task 16

Deploy Application With Vercel

- The Platform as a service
- Popular for deploying the JS Framework

The environment variable

- With the Vue framework with Vite
- The .env has been already installed
- But the rule has been added
- Only the env start with VITE can be used automatically
- To call it
 - `Import.meta.env.VITE_xxx`

What is required

- Update .env is boring
- The .env selection is provided
- .env, .env.local for the local use
- .env.development – for development purposes
- .env.production for the production stated
 - Used when npm run build is called

Don't Forget

- The CORS must be allowed for the new url is called
- Try Task 17

Security

- How to prevent the People who should not be able to access our resource

On Frontend side

- The component must be setup for the framework
- Different framework use different setting

Data Binding

- Previously is a one-way binding
 - Link data from JavaScript to show in the html
 - Receive the event call from html to call JavaScript function
- 2-Way binding
 - Link the data from the JavaScript to show in the HTML
 - Receive the Data from html to put in the variable in the JavaScript
 - With in single command

Form

- Html to receive the data
- Contain the input values with it name
- Contain the button with type submit
 - When click, the data will be sent to the action attribute target
 - If action is blank, the data is sent to the same page

```
1 <form action="/get-quote" method="POST">
2   <input type="text" name="first_name">
3   <input type="text" name="last_name">
4   <input type="email" name="email">
5   <button type="submit">Get Quote</button>
6   <button type="submit" formaction="add-another-quote">Add Quote</button>
7 </form>
```

V-model

- Link the data via the variable name

```
setup() {  
  const form = reactive({  
    name: '',  
    review: '',  
    rating: null  
  })  
  return {  
    form  
  }  
}
```

```
<label for="name">Name:</label>  
<input id="name" v-model="form.name">  
  
<label for="review">Review:</label>  
<textarea id="review" v-model="form.review"></textarea>  
  
<label for="rating">Rating:</label>  
<select id="rating" v-model.number="form.rating">  
  <option>5</option>  
  <option>4</option>  
  <option>3</option>  
  <option>2</option>  
  <option>1</option>  
</select>
```

V-model

- Force type
 - Default
 - The data receive from the input is String
 - So the type in JavaScript can be used

```
<select id="rating" v-model.number="rating">  
  <option>5</option>  
  <option>4</option>  
  <option>3</option>  
  <option>2</option>  
  <option>1</option>  
</select>
```

- Try Task 18

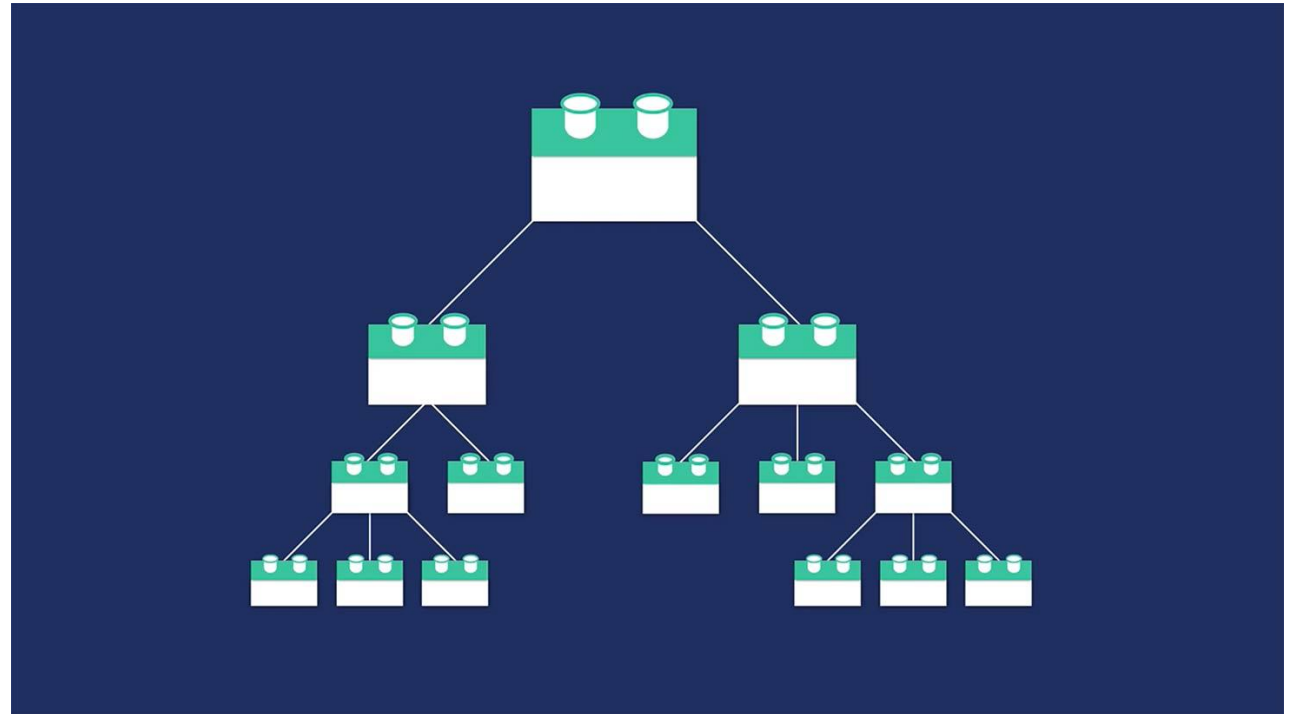
Frontend Component

Frontend the Component

- Separate the code (or the visualization as the components)
- Each component contains a similar structure
 - Different value
- Load the component to be used
- Reduce the number of element
- Reused the component
- Easy to update in the future

Components

- Each component(block)
 - May be composed with multiple components
- Can be “parent” component
 - Contains “child” components

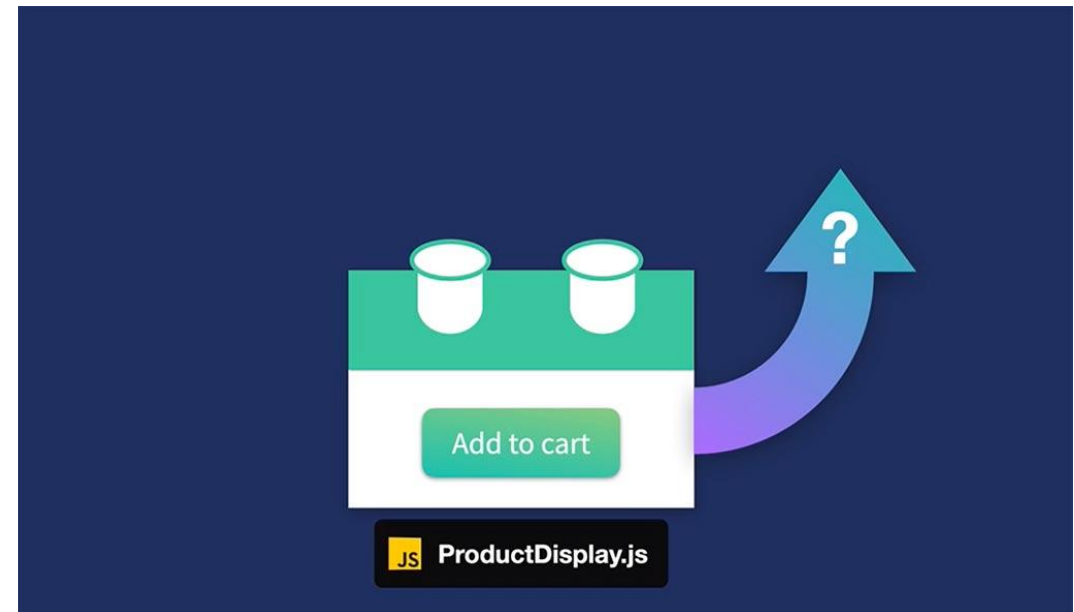


Component

- Create as view
- Import to the view
- The name of the import will be used as the HTML tag
- The prop is provided
- The emit is used to send the event back to be handled in the source code

Sending the events

- When the child component receive the event
- We may want to send the event to the parent
 - To update some variable in the parent component



What to do?

- When some event occurred
 - Send the event to someone else
 - Using emit command

```
<button class="button" :disabled='!inStock' @click="addToCart"
  :class="{disabledButton: !inStock}">Add To
  Cart</button>
```

```
function addToCart() {
  emit('add-to-cart')
}
```

```
setup(props, { emit }) {
```

emit

- Create the event
- Event name is the first input parameter
- Think like click
 - Event happen
 - Need some one to map handler for the event

Mapping the event handler

- Using the simple mapping

```
function addToCart() {  
  emit('add-to-cart')  
}
```

```
<product-display :premium="premium" @add-to-cart="updateCart"></product-display>
```

```
function updateCart() {  
  cart.value += 1  
}
```

We can send other information with event

- As the second parameters

```
function addToCart() {  
  |   emit('add-to-cart', variants.value[selectedVariant.value].id)  
  |  
}
```

- When handle the event, the input parameter are required
 - The Framework will inject the input parameter from the object sent by the event

```
<product-display :premium="premium" @add-to-cart="updateCart"></product-display>
```

```
function updateCart() {  
  |   cart.value += 1  
  |  
}
```

```
function updateCart(id) {  
  |   cart.value.push(id)  
  |  
}
```

BaseInput

- Just only the example
- You can update what ever you want

Putting the content in the component

```
<h3>Name & describe your event</h3>
```

```
<label>Title</label>
```

```
<input v-model="event.title" type="text" placeholder="Title" class="field" />
```

```
<BaseInput v-model="event.title" label="Title" placeholder="Title" />
```

Creating a component

components

- icons
- ▼ BaseInput.vue

```
<script setup lang="ts">
const modelValue = defineModel()
interface BaseInputProps {
  label: string
}
const props = withDefaults(defineProps<BaseInputProps>(), {
  label: ''
})
</script>
<template>
  <label v-if="label">
    {{ props.label }}
  </label>
  <input class="mb-6" v-bind="$attrs" v-model="modelValue" :placeholder="label" />
</template>
```

```
<label>Title</label>
<input v-model="event.title" type="text" placeholder="Title" class="field" />
```

Receiving the Data via label variable

- define internal interface type
- defineProps with the type

```
interface BaseInputProps {  
  label: string  
}  
const props = withDefaults(defineProps<BaseInputProps>(), {  
  label: ''  
})
```

- withDefaults for providing the default value
 - Use when sometime the input is not required

- To use

- In ts can use the name directly
- To get the text
 - Using the props variable name and . by the value name

```
<template>  
  <label v-if="label">  
    {{ props.label }}  
  </label>
```


modelValue

- modelValue
 - The target of the v-bind
 - It will link the data from the component
 - To the parent component and vice versa

modelValue = defineModel()

- Start from Vue 3.4
- Provide two-way binding from the parent and child component
- The data which will be linked with the v-model of the parent

```
const modelValue = defineModel()  
</script>  
<template>  
  <label v-if="label">  
    {{ props.label }}  
  </label>  
  <input class="mb-6" v-bind="$attrs" v-model="modelValue" :placeholder="label" />  
</template>
```

```
<BaseInput v-model="event.title" type="text" label="Title" />
```

The \$attrs

- Get all attributes of the component
 - Links to the component in the template

```
<template>
  <label v-if="label">
    | {{ props.label }}
  </label>
  <input class="mb-6" v-bind="$attrs" v-model="modelValue" :placeholder="label" />
</template>
```

```
<label>Title</label>
<input v-model="event.title" type="text" placeholder="Title" class="field" />
```

- In this case type attribute is bound
- Inject attributes directly to the input

All together

- Better code reader
- Easy to update

```
<h1>Create an event</h1>
<form @submit.prevent="saveEvent">
  <BaseInput v-model="event.category" type="text" label="Category" />
  <h3>Name & describe your event</h3>

  <BaseInput v-model="event.title" type="text" label="Title" />

  <BaseInput v-model="event.description" type="text" label="Description" />

  <h3>Where is your event?</h3>

  <BaseInput v-model="event.location" type="text" label="Location" />
```

```
<form @submit.prevent="saveEvent">
  <label>Category</label>
  <input v-model="event.category" type="text" placeholder="Category" class="field" />
  <h3>Name & describe your event</h3>

  <label>Title</label>
  <input v-model="event.title" type="text" placeholder="Title" class="field" />

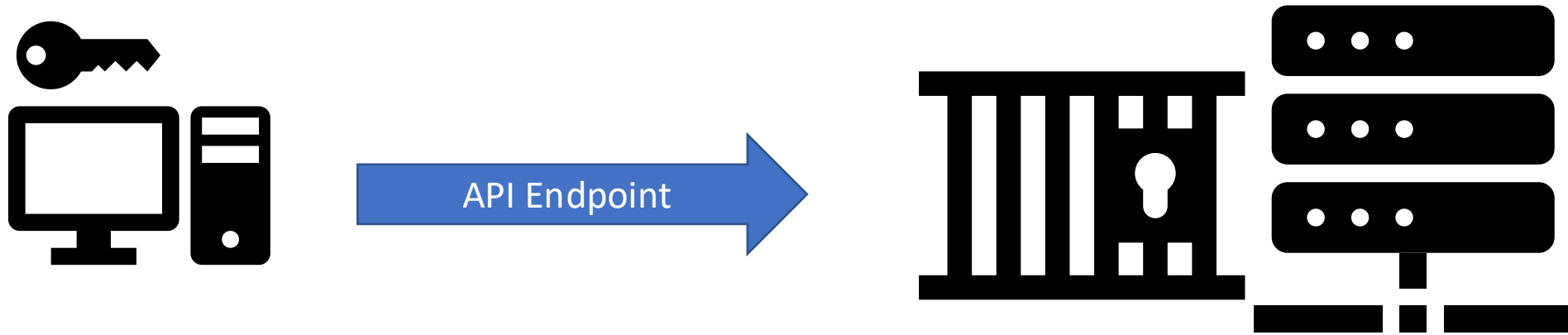
  <label>Description</label>
  <input v-model="event.description" type="text" placeholder="Description" class="field" />

  <h3>Where is your event?</h3>

  <label>Location</label>
  <input v-model="event.location" type="text" placeholder="Location" class="field" />
```

Try Task 19

On Frontend application



How to attach?

- Add the Token to the HTTP header

▼ Request Headers [view source](#)

Accept: application/json, text/plain, */*
Accept-Encoding: gzip, deflate, br
Accept-Language: en-US,en;q=0.9
Authorization: Bearer eyJhbGciOiJIUzUxMiJ9.eyJzdWIiOiJhZG1pbiIsImF1ZG11bmNlIjoid2VU0MjY5MjEzODg4NSwiZXhwIjoxNTQzMjk2OTM4fQ.kn-SQQCB7FqOCJH086i76EXgJWknYto2iNAb2IR'CdTgZgXpbtGKO1S69XfMob2xI8zw
Connection: keep-alive
Host: localhost:8080
Origin: http://localhost:4200
Referer: http://localhost:4200/list
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, 70.0.3538.102 Safari/537.36

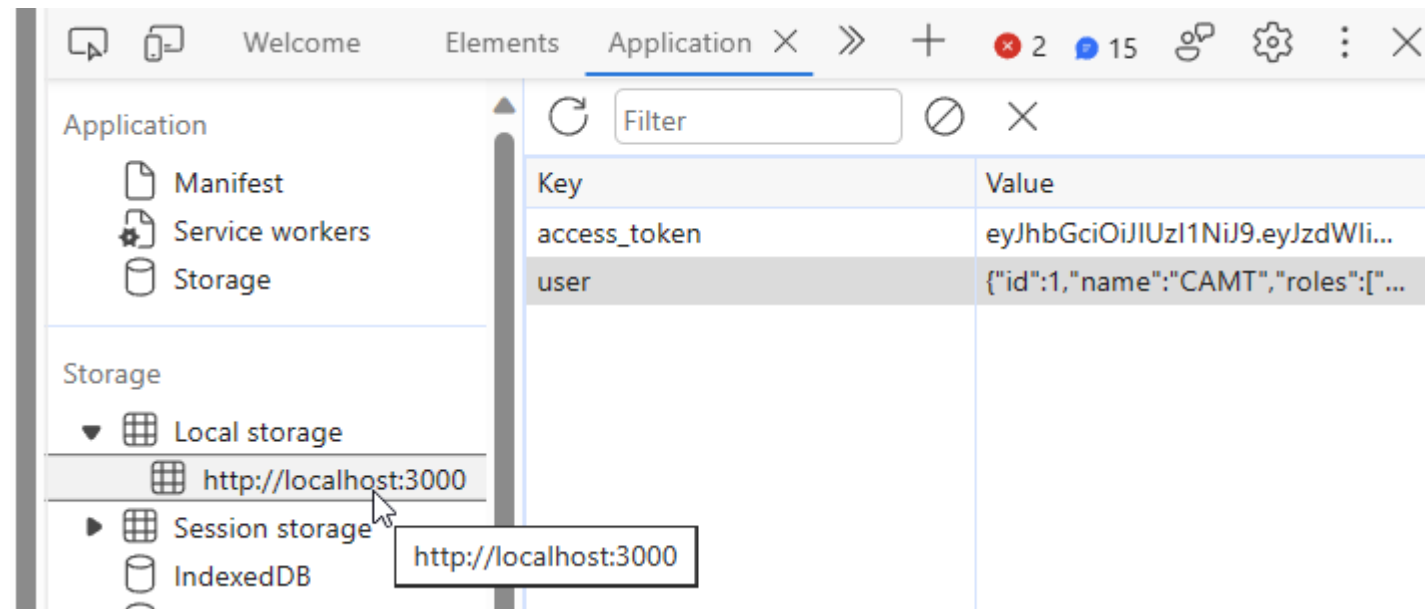
Authorize Bearer

- The common name of the authorization
- Map with the backend code

```
class JwtAuthenticationTokenFilter {  
  
    @Override  
    protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response, FilterChain chain)  
    {  
        String authToken = request.getHeader(this.tokenHeader);  
  
        if (authToken != null && authToken.startsWith("Bearer ")) {  
            authToken = authToken.substring(7);  
        }  
  
        String username = jwtTokenUtil.getUsernameFromToken(authToken);  
  
        // in the token and read it from it. It's up to you ;)  
        UserDetails userDetails = this.userDetailsService.loadUserByUsername(username);  
  
        // For simple validation it is completely sufficient to just check the token integrity. You don't h  
        // the database compellingly. Again it's up to you ;)  
        if (jwtTokenUtil.validateToken(authToken, userDetails)) {  
            UsernamePasswordAuthenticationToken authentication = new UsernamePasswordAuthenticationToken(us  
            authentication.setDetails(new WebAuthenticationDetailsSource().buildDetails(request));  
            logger.info("authenticated user " + username + ", setting security context");  
            SecurityContextHolder.getContext().setAuthentication(authentication);  
        }  
    }  
}
```


How we deal with the returns data

- Keep it in the LocalStorage
- HTML5 features
 - per origin (per domain and protocol). All pages, from one origin, can store and access the same data.



Provide the Pinia to hold the data

- Try Task 20

```
export const useAuthStore = defineStore('auth', {
  state: () => ({
    token: null as string | null
  }),
  actions: {
    login(email: string, password: string) {
      console.log('login')
      return apiClient
        .post('/api/v1/auth/authenticate', {
          username: email,
          password: password
        })
        .then((response) => {
          this.token = response.data.access_token
          axios.defaults.headers.common['Authorization'] = `Bearer ${this.token}`
          return response
        })
    }
  }
})
```

Vee validate

- The validation component for Vue
- Client-Side validation
 - Return the user immediate feedback
 - Increase user experience
 - Save resources by eliminating the invalid data

Vee validate

- The latest version is v.4
- [Validation | vee-validate \(logaretm.com\)](https://logaretm.com/vee-validate/)
- Provide 2 component for validation
 - Form
 - Field
- Another component for the error message
 - Error message

Yup api

- Popular simple and powerful data validation library
- [jquense/yup: Dead simple Object schema validation \(github.com\)](https://github.com/jquense/yup)
- Provide the validation for the text field
 - String
 - The length, required
 - mixed
 - Check for all types
 - date
 - array
- Provide the error message under the schema

To set up the validation

- Set up the schema
 - Define the name of field
 - Return the schema as the nested

```
import { useField, useForm } from 'vee-validate'

import * as yup from 'yup'

const validationSchema = yup.object({
  email: yup.string().required('The email is required'),
  password: yup.string().required('The password is required')
})
```

```
const { errors, handleSubmit } = useForm({
  validationSchema,
  initialValues: {
    email: '',
    password: ''
  }
})
```

```
const { value: email } = useField<string>('email')
const { value: password } = useField<string>('password')

<form class="space-y-6" @submit.prevent="onSubmit">
  <div>
    <label for="email" class="block text-sm font-medium leading-6 text-gray-900">Email address</label>
    <div class="mt-2">
      <input id="email" name="email" type="email" class="block w-full rounded-md border-0 py-1.5 text-gray-900">
      <input type="text" v-model="email" :error="errors['email']">
    </div>
    <div>
      <div class="flex items-center justify-between">
        <label for="password" class="block text-sm font-medium leading-6 text-gray-900">Password</label>
        <div class="text-sm">
          <a href="#" class="font-semibold text-indigo-600 hover:text-indigo-500">Forgot password?</a>
        </div>
      </div>
      <input type="password" v-model="password" :error="errors['password']">
    </div>
  </div>
</form>
```

Using the Form, and Field component

- Using from the Vee validate component

```
const { errors, handleSubmit } = useForm({  
  validationSchema,  
  initialValues: {  
    email: '',  
    password: ''  
  }  
})
```

```
const onSubmit = handleSubmit((values) => {  
  authStore.login(values.email, values.password)  
    .then(() => {  
      router.push({ name: 'event-list' })  
    }).catch(() => {  
      messageStore.updateMessage('could not login')  
      setTimeout(() => {  
        messageStore.resetMessage()  
      }, 3000)  
    })  
})  
  
<form class="space-y-6" @submit.prevent="onSubmit">
```

- Assigned to the schema

```
const validationSchema = yup.object({  
  email: yup.string().required('The email is required').email('Input must be an email.'),  
  password: yup.string().required('The password is required').min(6, 'The password must be at least 6 characters.')  
})
```

The token

- Provided in the response data
- Add in the state

To set the header

```
actions: {  
  login(email: string, password: string) {  
    return apiClient  
      .post('/api/v1/auth/authenticate', {  
        username: email,  
        password: password  
      })  
      .then((response) => {  
        this.token = response.data.access_token  
        localStorage.setItem('access_token', this.token as string)  
        return response  
      })  
  }  
}
```

General

Request URL: http://localhost:8080/events?_limit=3&_page=1

Request Method: GET

Status Code:  200

Remote Address: [::1]:8080

Referrer Policy: strict-origin-when-cross-origin

Request Headers

[View source](#)

Accept: application/json

Accept-Encoding: gzip, deflate, br

Accept-Language: en-US,en;q=0.9,th;q=0.8,de;q=0.7

Authorization: Bearer eyJhbGciOiJIUzUxMiJ9.eyJzdWIiOiJhZG1pbmN1Ijoid2ViIiwiaWF0Ij0iYzZMMDQ0MzY3MywiZXhwIjoxNjMzOTY1MjQzZQ.2CJ0fYEN4r3YCsmehtPvTvLdjX19zXdn7S2Fxf1BZUWUdGet5aNMoiIL2NPFfe2KNVO-70hg

Connection: keep-alive

Host: localhost:8080

Origin: http://localhost:3000

Referer: http://localhost:3000/

sec-ch-ua: "Chromium";v="94", "Microsoft Edge";v="94", ";Not A Brand";v="99"

sec-ch-ua-mobile: ?0

sec-ch-ua-platform: "Windows"

Sec-Fetch-Dest: empty

Sec-Fetch-Mode: cors

Sec-Fetch-Site: same-site

User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like 94.0.4606.71 Safari/537.36 Edg/94.0.992.38)

Need to add in every call

- So boring
- Not good design pattern
 - A lot of duplicate code
- Using interceptor
 - Add for all request
 - Create the Singleton apiClient
 - Set the interceptor and use everywhere

Refactoring the code

AxiosClient.ts

```
import axios from 'axios'
const apiClient = axios.create({
  baseURL: import.meta.env.VITE_BACKEND_URL,

  headers: {
    Accept: 'application/json',
    'Content-Type': 'application/json'
  }
})
export default apiClient
```

OrganizationService.ts

```
import type { AxiosResponse } from 'axios'
import apiClient from './AxiosClient'
import type { EventOrganizer } from '@type'

export default {
  getOrganizers(): Promise<AxiomResponse<EventOrganizer[]>> {
    return apiClient.get<EventOrganizer[]>(`/organizers`)
  }
}
```

EventService.ts

```
import apiClient from './AxiosClient'
import type { AxiosResponse } from 'axios'
import type { EventItem } from '@type'

export default {
  getEvent(perPage: number, page: number): Promise<AxiomResponse<EventItem[]>> {
    return apiClient.get<EventItem[]>(`/events?_limit=${perPage}&_page=${page}`)
  },
}
```

Adding Interceptor

AxiosInterceptorSetup.ts

```
import axios from 'axios';
import apiClient from '@services/AxiosClient.js'
apiClient.interceptors.request.use(
  (request) => {
    const token = localStorage.getItem('access_token')
    console.log('token', token)
    if (token) {
      request.headers['Authorization'] = 'Bearer ' + token
    }
    return request
  },
  (error) => {
    return Promise.reject(error)
  }
)
```

main.ts

```
import '@services/AxiosInrceptorSetup.ts'
```

Try Task 21 22 23 24

To Logout

- Remove the content in the local storage, and the state

```
actions: {  
  logout() {  
    console.log('logout')  
    this.token = null  
    this.user = null  
    localStorage.removeItem('access_token')  
    localStorage.removeItem('user')  
  },  
}
```

Reactive Object

- The normal data when it change
 - The render is not change
- Only the reactive object changed
 - The render will be changed
- Using the getter methods
 - Which is similar to the computed

```
getters: {  
  isAuthenticated(): boolean {  
    | return !!this.token  
  },  
  currentUserName(): string {  
    | return this.user?.name || ''  
  }  
},
```

When the data is changed

- The related data is render

```
getters: {
  isAuthenticated(): boolean {
    return !!this.token
  },
  currentUserName(): string {
    return this.user?.name || ''
  }
},

login(email: string, password: string) {
  return apiClient
    .post('/api/v1/auth/authenticate', {
      username: email,
      password: password
    })
    .then((response) => {
      this.token = response.data.access_token
      this.user = response.data.user
      localStorage.setItem('access_token', this.token as string)
      localStorage.setItem('user', JSON.stringify(this.user))

      return response
    })
}
```

App.vue

```
you, 7 days ago | 2 authors (you and others)
<script setup lang="ts">
import { RouterLink, RouterView } from 'vue-router'
import { useMessageStore } from '@stores/message'
import { useAuthStore } from './stores/auth';
import { storeToRefs } from 'pinia'
import { useRouter } from 'vue-router';
const store = useMessageStore()
const authStore = useAuthStore()

<nav class="flex">
  <ul v-if="!authStore.currentUserName" class="flex navbar-nav ml-auto">
    <li class="nav-item px-2">
      <router-link to="/register" class="nav-link">
        <font-awesome-icon icon="user-plus" /> Sign Up
      </router-link>
    </li>
    <li class="nav-item px-2">
      <router-link to="/login" class="nav-link">
        <font-awesome-icon icon="sign-in-alt" /> Login
      </router-link>
    </li>
  </ul>
  <ul v-if="authStore.currentUserName" class="flex navbar-nav ml-auto">
```


The SVG Icon

- Provide the free icon
- Setting with the Vue
- [Getting Started with Vue.js - Docs – Pictogrammers](#)
- Checking the value to show the image
- Try Task 10

Everything keep Pinia

- For the reactive management
- When refresh
- Everything will reset
- As single page application

Every time it is reloaded

- The data is set to default

```
export const useAuthStore = defineStore('auth', {  
  state: () => ({  
    token: null as string | null,  
    user: null as EventOrganizer | null  
  }),  
})
```

- Pinia is the in-memory state management
 - When reload the data, all the in memory are reset

To maintain the status even the page is reloaded

- Reload data when the app is reloaded

```
const token = localStorage.getItem('token')
const user = localStorage.getItem('user')
if (token && user) {
  |   authStore.reload(token, JSON.parse(user))
}else{
  |   authStore.logout()
}

reload(token: string, user: EventOrganizer) {
  |   this.token = token
  |   this.user = user
}
```

- Task 11

How to protected people in frontend

- Checking the role
- Using pinia for the global checking

```
getters: {  
  currentUserName(): string {  
    return this.user?.username || ''  
  },  
  isAdmin(): boolean {  
    return this.user?.roles.includes('ROLE_ADMIN') || false  
  }  
}
```

Protected By Hiding the Menu

- User cannot access via Link
- V-if to select what to show

```
.....  
    <RouterLink :to="{ name: 'event-list-view' }">Event</RouterLink> |  
    <RouterLink :to="{ name: 'about' }">About</RouterLink>  
    <span v-if="authStore.isAdmin"> | <RouterLink to="/admin">Admin</RouterLink></span>  
  </nav>
```

Blocking from url

- Some people can type the URL directly
- So the resource should be checked before entering the page
- `beforeEach` or `beforeEnter` is used

```
beforeEnter: (to) => {  
  const id = parseInt(to.params.id as string)  
  const eventStore = useEventStore()  
  return eventService  
    .getEvent(id)  
    .then((response) => {  
      // need to setup the data for the event  
      eventStore.setEvent(response.data)  
    })  
    .catch((error) => {  
      if (error.response && error.response.status === 404) {  
        return {  
          name: '404-resource-view',  
          params: { resource: 'event' },  
        }  
      } else {  
        return { name: 'network-error-view' }  
      }  
    })  
}
```

Nodejs Package management

- Npm – Node Package Manager
 - Modular development
 - Download package with the transitive to the project
 - Contain command line for node execution

The package descriptor

- package.json

```
{  
  "name": "lab01",  
  "version": "0.0.0",  
  "scripts": {  
    "ng": "ng",  
    "start": "ng serve",  
    "build": "ng build",  
    "test": "ng test",  
    "lint": "ng lint",  
    "e2e": "ng e2e"  
  },  
}
```

```
  "dependencies": {  
    "@angular/animations": "^8.2.4",  
    "@angular/cdk": "^8.1.4",  
    "@angular/common": "~8.2.0",  
    "@angular/compiler": "~8.2.0",  
    "@angular/core": "~8.2.0",  
    "@angular/forms": "~8.2.0",  
    "@angular/material": "^8.1.4",  
    "@angular/platform-browser": "~8.2.0",  
    "@angular/platform-browser-dynamic": "~8.2.0",  
    "@angular/router": "~8.2.0",  
    "bootstrap": "^4.3.1",  
    "hammerjs": "^2.0.8",  
    "jquery": "^3.4.1",  
    "popper.js": "^1.15.0",  
    "rxjs": "~6.4.0",  
    "tslib": "^1.10.0",  
    "zone.js": "~0.9.1"  
  },  
}
```

```
  "devDependencies": {  
    "@angular-devkit/build-angular": "~0.802.1",  
    "@angular/cli": "~8.2.1",  
    "@angular/compiler-cli": "~8.2.0",  
    "@angular/language-service": "~8.2.0",  
    "@types/node": "~8.9.4",  
    "@types/jasmine": "~3.3.8",  
    "@types/jasminewd2": "~2.0.3",  
    "codemirror": "^5.0.0",  
    "jasmine-core": "~3.4.0",  
    "jasmine-spec-reporter": "~4.2.1",  
    "karma": "~4.1.0",  
    "karma-chrome-launcher": "~2.2.0",  
    "karma-coverage-istanbul-reporter": "~2.0.1",  
    "karma-jasmine": "~2.0.1",  
    "karma-jasmine-html-reporter": "^1.4.0",  
    "protractor": "~5.4.0",  
    "ts-node": "~7.0.0",  
    "tslint": "~5.15.0",  
    "typescript": "~3.5.3"  
  }  
}
```

Repository

- Package are downloaded to node-package
 - Package is not added to the code repository
 - Reduce the amount of data to be transferred to the code repository
 - Rarely change after upload
 - .gitignore

```
# compiled output
/dist
/tmp
/out-tsc

# dependencies
/node_modules
```

Npm install

- Without the parameter
 - Read the dependency in the package.json
 - Download them
- With parameter
 - Download individual package to the project
 - Using --save, --save-dev
 - Update the package.json

Version control

- `~version` "Approximately equivalent to version"
 - Match the most recent minor version (the middle number). `~1.2.3` will match all 1.2.x versions but will miss 1.3.0
- `^version` "Compatible with version"
 - More relaxed
 - The most recent major version (the first number). `^1.2.3` will match any 1.x.x release including 1.3.0, but will hold off on 2.0.0.
- <https://docs.npmjs.com/misc/semver#prerelease-identifiers>

Script management

- Provide the script for use the node component
- The component which install with `-g`
 - Can used directly
- Otherwise
 - Setup via the script
 - `npm run "script name"`

```
"scripts": {  
  "ng": "ng",  
  "start": "ng serve",  
  "build": "ng build",  
  "test": "ng test",  
  "lint": "ng lint",  
  "e2e": "ng e2e"  
},
```

Working with the NPM

- Finding the component
- Install
- Link
- Use

Finding the Component

- npmjs.com

vue-media-upload

2.1.2 • Public • Published 6 months ago

Readme

Code

Beta



0 Dependencies

npm v2.1.2

downloads

1.6k/month

license

MIT

minzipped size

746 B

vue-media-upload

 **vue-media-upload** is a Vuejs package that handle multiple images upload and preview.

- Can check for source code

Install

```
> npm i vue-media-upload
```

Repository

 github.com/saimow/vue-media-upload

Homepage

 [github.com/saimow/vue-media-upload...](https://github.com/saimow/vue-media-upload)

Weekly Downloads

306



Version

2.1.2

License

MIT

Unpacked Size

140 kB

Total Files

11

Issues

4

Pull Requests

4

Install

- Using npm install command

```
> npm i vue-media-upload
```

- Dependency will be added, in node_modules folder

```
> vue  
> vue-eslint-parser  
> vue-media-upload  
> vue-router  
> vue-template-compiler
```


If the project is clone

- Npm install
 - Download all dependency from

```
npm package-lock.json  
npm package.json
```

- Make the workspace are the same even in the different computer

Still other package management tools

- Depending on your programming language

QA 😊